Security Awareness Phishing Simulation

Apex Planet Cybersecurity Internship

Prepared by	Intern – Apex Planet
Date	April 26, 2026
Tool Stack	Gophish v0.12.1, MailHog, Kali Linux
Environment	Isolated Lab (127.0.0.1)
Classification	Confidential – Internal Use Only
Report Version	2.0

Table of Contents

1. Executive Summary
2. Project Planning
    2.1 Objectives
    2.2 Scope and Constraints
    2.3 Tools and Technologies
    2.4 Project Timeline
    2.5 Network and Environment Diagram
3. Methodology
4. Implementation
    4.1 Phase 1 – Infrastructure Setup
    4.2 Phase 2 – Reconnaissance
    4.3 Phase 3 – Campaign Configuration
    4.4 Phase 4 – Campaign Execution and Results
5. Findings and Evidence
6. Mitigation Strategies
7. Incident Response Simulation
    7.1 Detecting the Attack
    7.2 Containment and Eradication
    7.3 Post-Incident Report
8. Conclusion
9. Appendix – All Screenshots

1. Executive Summary

This report documents a phishing simulation exercise carried out as part of the Apex Planet Cybersecurity Internship. The goal was to go through a complete simulated phishing attack from beginning to end — setting up the infrastructure, crafting the email, launching the campaign, and then looking at how an organisation would detect and respond to it.

The tools used were Gophish v0.12.1 (phishing framework) and MailHog (fake SMTP server), both running on a local Kali Linux machine. Everything was kept to the loopback address (127.0.0.1) so no real users or internet systems were involved at any point.

The second campaign run (Campaign 3) showed successful end-to-end phishing: the email was sent, opened, the phishing link was clicked, and the target submitted data on the fake login page. This demonstrates exactly the kind of attack a real organisation needs to defend against.

Metric	Campaign 1 (Test)	Campaign 3 (Full Simulation)
Email Sent	1	1
Email Opened	0	1
Link Clicked	0	1
Data Submitted	0	1
Email Reported	0	0

2. Project Planning

2.1 Objectives

The main objectives I set out for this project were:

– Set up a working phishing simulation lab using free, open-source tools.

– Go through all the steps an attacker would take: reconnaissance, infrastructure setup, email crafting, and campaign launch.

– Verify that the phishing email reaches the target and that tracking (opens, clicks, submissions) works correctly.

– Capture evidence at every step using screenshots.

– Understand how an organisation would detect and respond to this kind of attack.

– Suggest practical defences that would stop or reduce the impact of a real phishing attack.

2.2 Scope and Constraints

Item	Details
Environment	Fully isolated – localhost only (127.0.0.1), no internet traffic
Target	One synthetic test user: Test User <test@test.com>
Phishing server	Gophish admin on :3333, listener on :80
Email server	MailHog in Docker – SMTP on :1025, web UI on :8025
Operating system	Kali Linux
Out of scope	All real users, production systems, and internet infrastructure
Legal basis	Controlled internship lab exercise only

2.3 Tools and Technologies

Tool	Version	What I used it for
Gophish	v0.12.1	Running the phishing campaign – emails, landing pages, tracking
MailHog	Latest (Docker)	Catching all outgoing test emails so they go nowhere real
Docker	System default	Running MailHog in a container
Kali Linux	Rolling release	Main operating system for the whole exercise
Firefox	Latest	Accessing the Gophish dashboard and MailHog inbox
wget / unzip	System	Downloading and extracting the Gophish binary

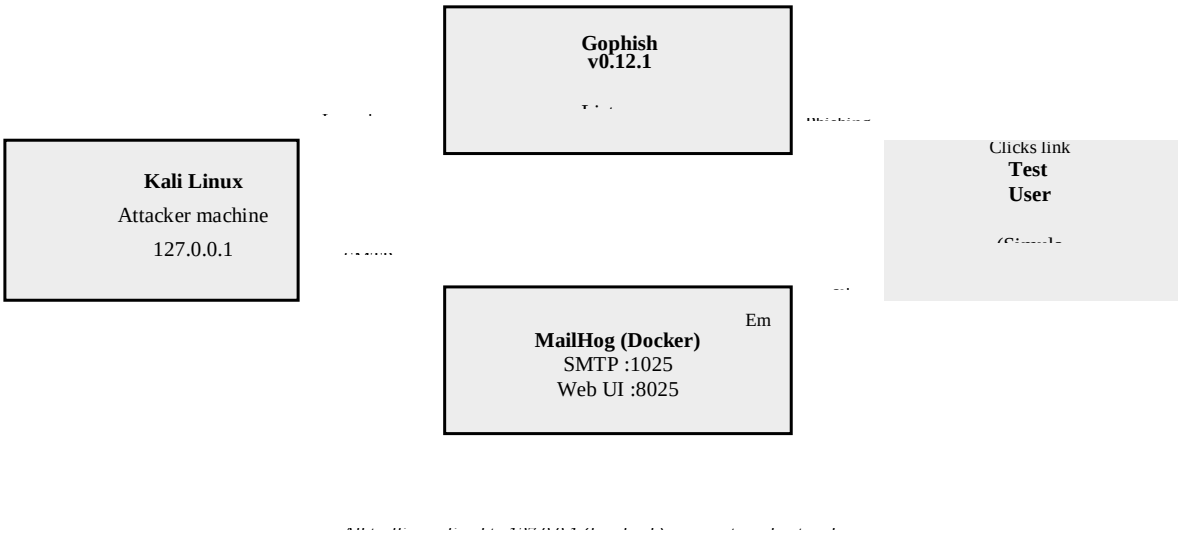
2.4 Project Timeline

Phase	Task	Time taken	Approx. date/time
1	Download and run MailHog via Docker	10 min	Mar 19, 05:45

2	Download, extract, and start Gophish	15 min	Mar 19, 05:45–06:00
3	Create Sending Profile (MailHog SMTP)	10 min	Mar 19, 06:02
4	Create Landing Page (Import Site)	15 min	Mar 19, 06:16
5	Create Email Template (Password Reset)	20 min	Mar 19, 06:26
6	Add Users & Groups (Test Group)	10 min	Mar 19, 06:27
7	Send test email and verify in MailHog	10 min	Mar 19, 06:27
8	Launch Campaign 1 and capture initial results	15 min	Mar 19, 06:30
9	Launch full Campaign 3 – open, click, submit	20 min	Mar 19, 06:47–06:48
10	Write up this report	~3 hrs	Mar 19–21, 2026

2.5 Network and Environment Diagram

The diagram below shows how the different components of the lab connected to each other. Everything runs on the same machine using loopback addresses — there is no external network traffic.



Network diagram – isolated lab environment

3. Methodology

I followed a standard phishing simulation methodology, loosely based on NIST SP 800-115 and the MITRE ATT&CK; framework (mainly T1566 – Phishing). The exercise was split into five phases:

Phase 1 – Reconnaissance
Figuring out the target, what pretext would work, and what the email should look like. In a real engagement this would involve OSINT. Here I created a synthetic target.

Phase 2 – Infrastructure Setup
Deploying MailHog and Gophish, then configuring the sending profile so Gophish could actually send emails through the fake SMTP server.

Phase 3 – Weaponisation
Building the phishing landing page (cloned login form) and the email template (Password Reset pretext with an urgent subject line and a link to the phishing page).

Phase 4 – Execution
Launching the campaign, watching the dashboard in real time, and verifying the email arrived in MailHog. Then running a second campaign to demonstrate opening, clicking, and submitting credentials on the fake page.

Phase 5 – Analysis and Reporting
Collecting all the evidence, writing up findings, and putting together mitigation recommendations and an incident response walkthrough.

4. Implementation

4.1 Phase 1 – Infrastructure Setup

I started by setting up the two main tools. First I launched MailHog in Docker so that any email Gophish sends gets caught locally instead of actually going anywhere. Then I downloaded and started Gophish.

Setting up MailHog:

```
sudo docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Port 1025 is the SMTP port that Gophish sends to. Port 8025 is the web interface where you can see all captured emails.

Setting up Gophish:

```
wget https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-v0.12.1-linux-64bit.zip unzip gophish-v0.12.1-linux-64bit.zip chmod +x gophish sudo ./gophish
```

On to http://127.0.0.1:80.

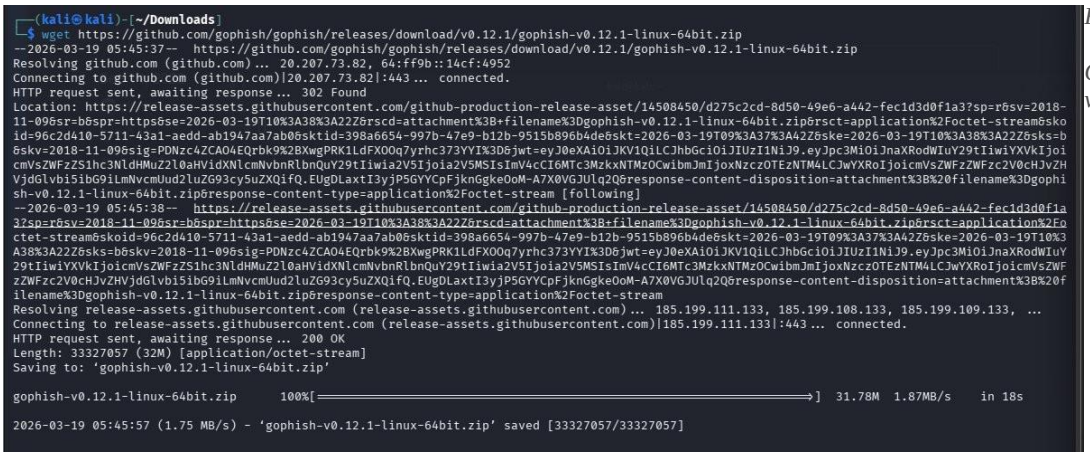


Figure 1 – Downloading Gophish v0.12.1 via wget

```
┌──(kali㉿kali)-[~/Downloads]
└─$ unzip gophish-v0.12.1-linux-64bit.zip
Archive:  gophish-v0.12.1-linux-64bit.zip
  inflating: gophish
   creating: static/js/dist/
   creating: static/js/dist/app/
  inflating: static/js/dist/app/sending_profiles.min.js
  inflating: static/js/dist/app/campaign_results.min.js
  inflating: static/js/dist/app/gophish.min.js
  inflating: static/js/dist/app/campaigns.min.js
  inflating: static/js/dist/app/autocomplete.min.js
  inflating: static/js/dist/app/settings.min.js
  inflating: static/js/dist/app/users.min.js
  inflating: static/js/dist/app/webhooks.min.js
  inflating: static/js/dist/app/dashboard.min.js
  inflating: static/js/dist/app/passwords.min.js
  inflating: static/js/dist/app/templates.min.js
  inflating: static/js/dist/app/groups.min.js
  inflating: static/js/dist/app/landing_pages.min.js
  inflating: static/js/dist/vendor.min.js
   creating: static/js/src/vendor/ckeditor/
  inflating: static/js/src/vendor/ckeditor/CHANGES.md
   creating: static/js/src/vendor/ckeditor/skins/
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_ie.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_gecko.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/icons.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/readme.md
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/refresh.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/arrow.png
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/
 extracting: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/refresh.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/lock.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/lock-open.png
 extracting: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/close.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/lock.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/lock-open.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/close.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/spinner.gif
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/icons_hidpi.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_ie.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_iequirks.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_iequirks.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_ie8.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_ie8.css
  inflating: static/js/src/vendor/ckeditor/LICENSE.md
   creating: static/js/src/vendor/ckeditor/plugins/
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/
  inflating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/specialchar.js
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/lang/
  inflating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/lang/zh-cn.js
```

Figure 2 – Extracting the archive

```
┌──(kali㉿kali)-[~/Downloads]
└─$ chmod +x gophish

┌──(kali㉿kali)-[~/Downloads]
└─$ ls -la
total 190660
drwxr-xr-x  5 kali kali     4096 Mar 19 05:47 .
drwx------ 30 kali kali     4096 Mar 19 05:45 ..
-rw-rw-r--  1 kali kali      940 Mar 15 09:17 cacert.der
-rw-r--r--  1 kali kali      460 Sep 14  2022 config.json
drwxr-xr-x  4 kali kali     4096 Sep 14  2022 db
-rwxr-xr-x  1 kali kali 21525728 Sep 14  2022 gophish
-rw-rw-r--  1 kali kali 33327057 Sep 14  2022 gophish-v0.12.1-linux-64bit.zip
-rw-r--r--  1 kali kali     1115 Sep 14  2022 LICENSE
-rw-rw-r--  1 kali kali 69359018 Mar 10 03:19 Nessus-10.11.3-debian10_amd64.deb
-rw-rw-r--  1 kali kali 70971470 Mar 10 08:08 Nessus-10.11.3-ubuntu1604_amd64.deb
-rw-r--r--  1 kali kali     3285 Sep 14  2022 README.md
drwxrwxr-x  7 kali kali     4096 Mar 19 05:47 static
drwxr-xr-x  2 kali kali     4096 Sep 14  2022 templates
-rw-r--r--  1 kali kali        7 Sep 14  2022 VERSION

┌──(kali㉿kali)-[~/Downloads]
└─$ sudo ./gophish
time="2026-03-19T05:50:11-04:00" level=warning msg="No contact address has been configured."
time="2026-03-19T05:50:11-04:00" level=warning msg="Please consider adding a contact_address entry in your config.json"
goose: migrating db environment 'production', current version: 0, target: 20220321133237
OK    20160118194630_init.sql
OK    20160131153104_0.1.2_add_event_details.sql
OK    20160211211220_0.1.2_add_ignore_cert_errors.sql
OK    20160217211342_0.1.2_create_from_col_results.sql
OK    20160225173824_0.1.2_capture_credentials.sql
OK    20160227180335_0.1.2_store-smtp-settings.sql
OK    20160317214457_0.2_redirect_url.sql
OK    20160605210903_0.2_campaign_scheduling.sql
OK    20170104220731_0.2_result_statuses.sql
OK    20170219122503_0.2.1_email_headers.sql
```

Figure 3 – Running Gophish for the first time; database migrations applied

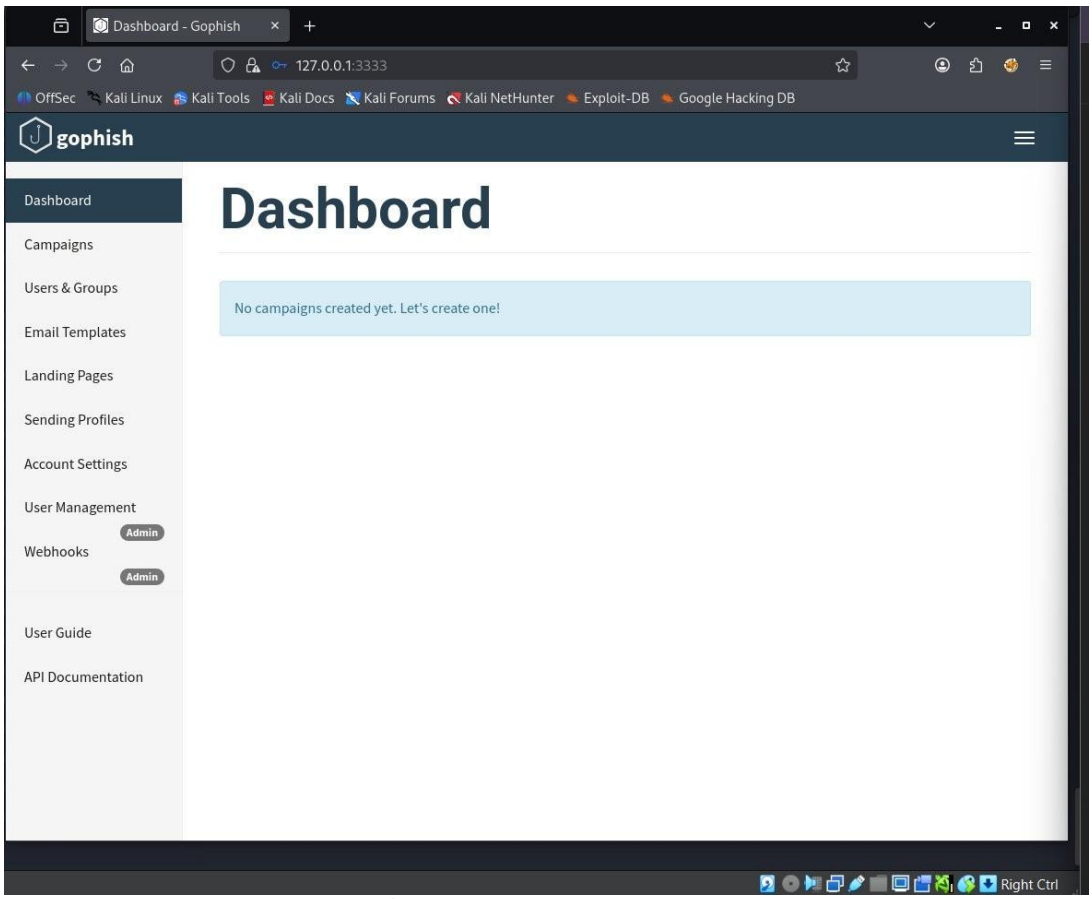


Figure 4 – Gophish admin dashboard after login

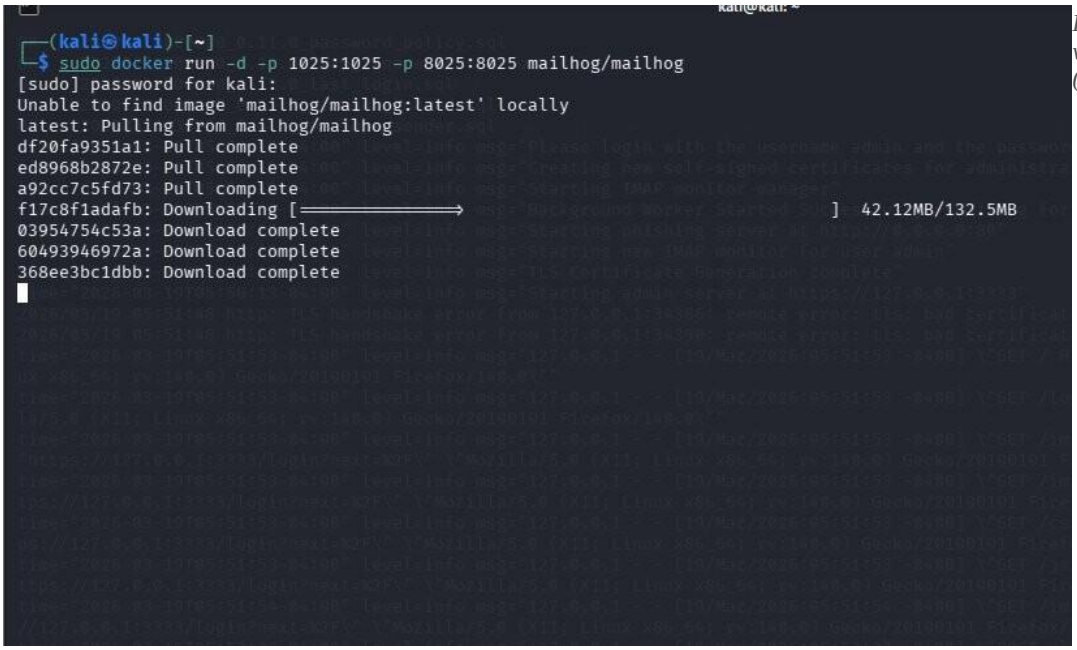


Figure 5 – MailHog web UI on first load (empty inbox)

4.2 Phase 2 – Reconnaissance

In a real engagement, reconnaissance means gathering information about targets using OSINT techniques — finding email addresses, job titles, names from LinkedIn, company websites, breach databases, and so on. For this lab I skipped that and just defined a synthetic target to keep things simple:

Field	Value
First name	Test
Last name	User
Email	test@test.com
Position	Employee (generic)
Pretext chosen	"Password Reset" – high urgency, works for any role

MITRE ATT&CK; reference: T1589 (Gather Victim Identity Information), T1598.003 (Spearphishing Link)

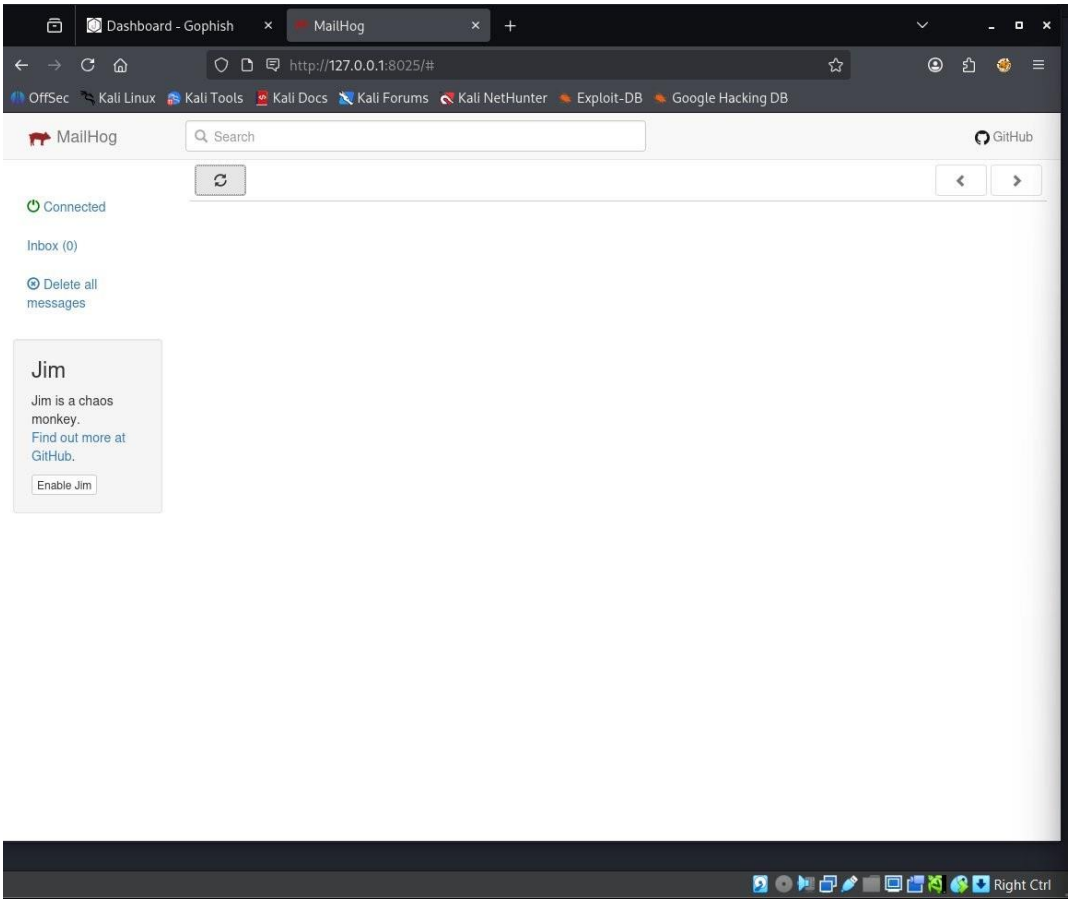
4.3 Phase 3 – Campaign Configuration

Sending Profile

I created a Sending Profile called "MailHog" that points Gophish at the local MailHog SMTP server. This means no real emails ever leave the machine.

Figu

```
Name: MailHog SMTP From: Test User Host: 127.0.0.1:1025 Username: (none) Password:
(none) Ignore Certificate Errors: Yes
```



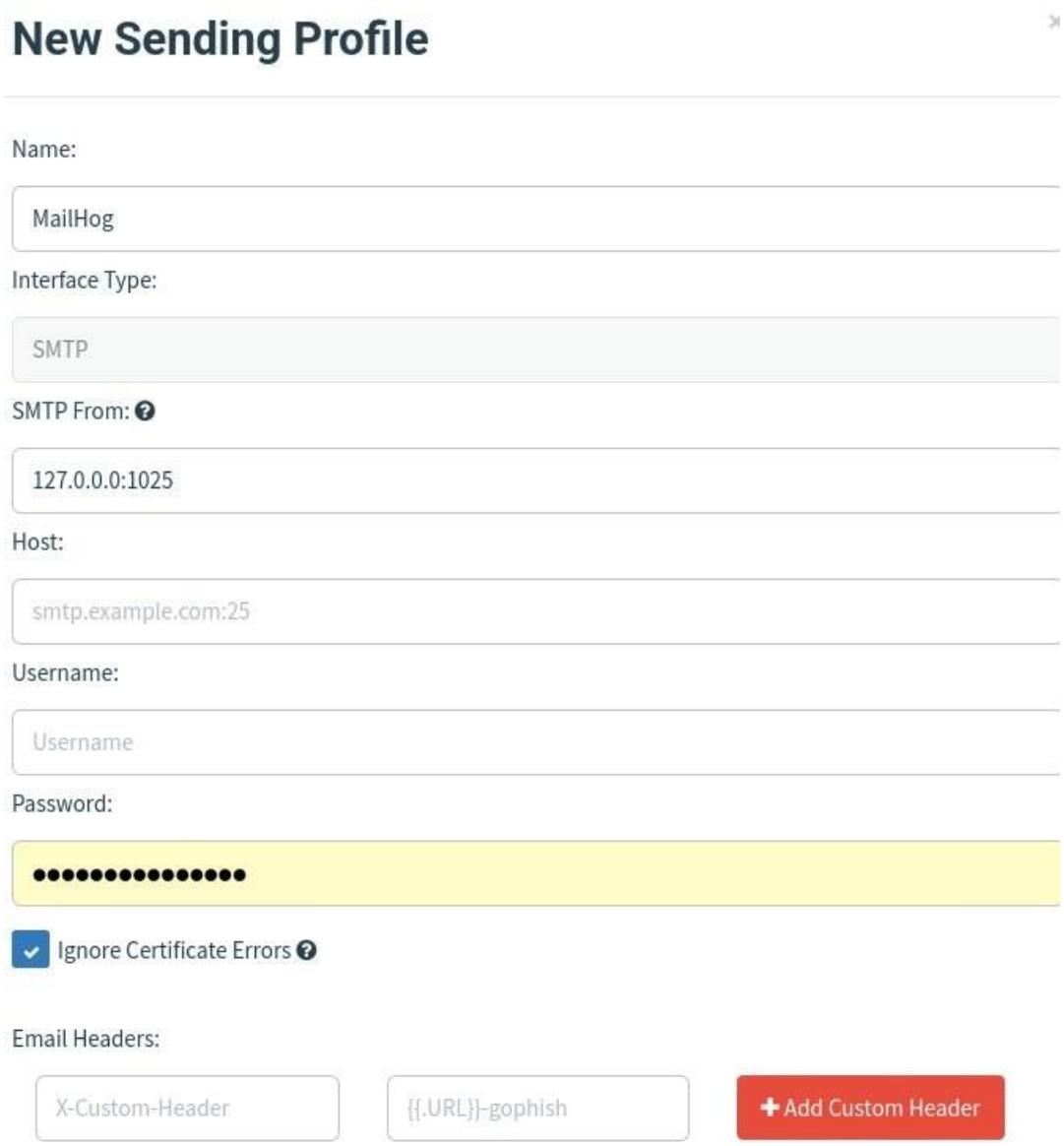


Figure 7 – Sending Profile form (lower section)

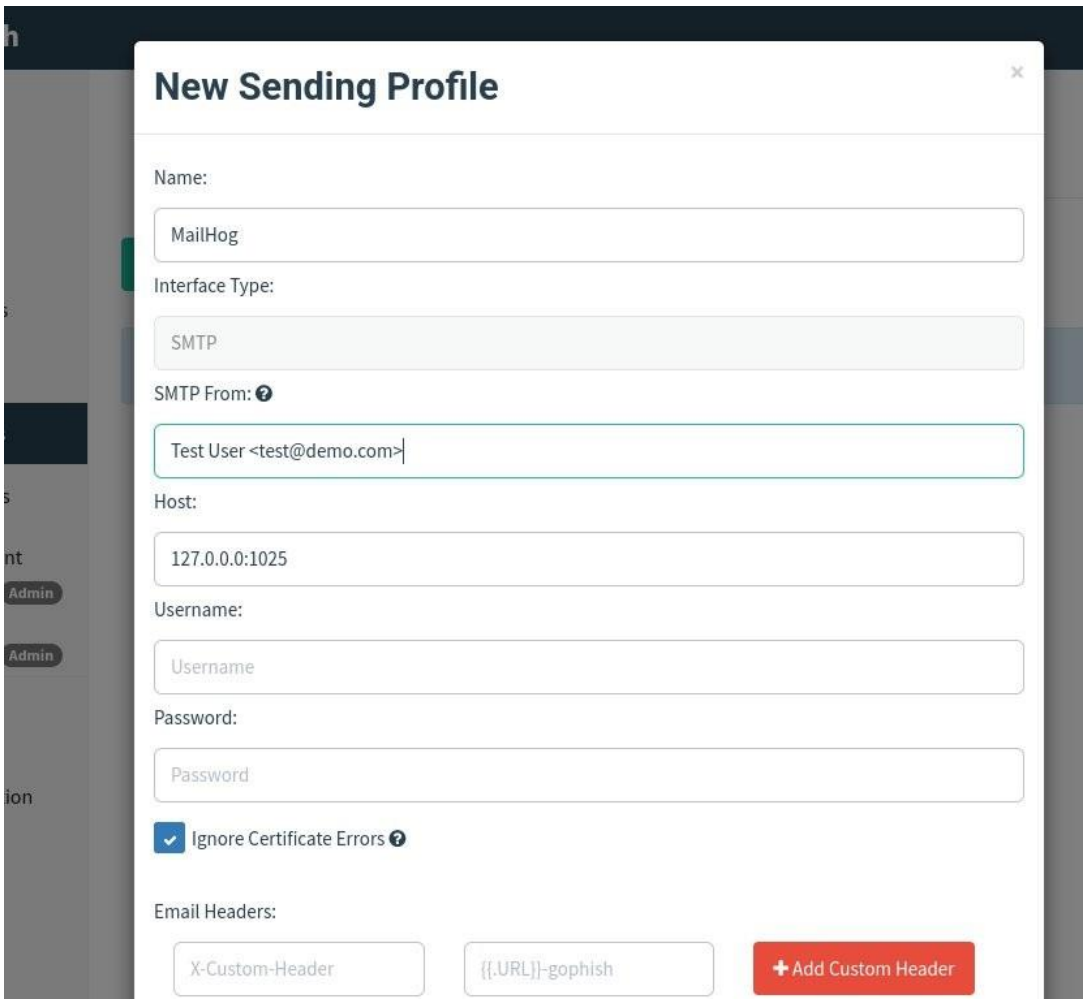


Figure 8 – Sending Profile saved

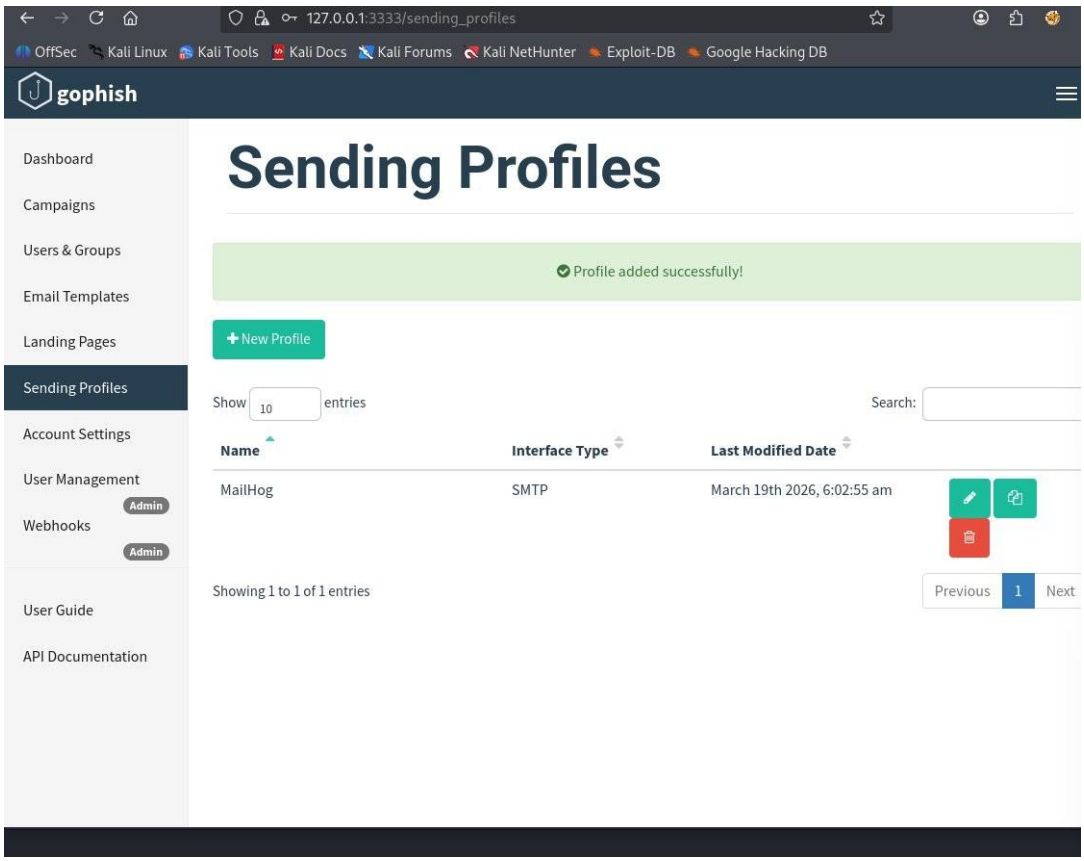


Figure 9 – Sending Profiles list confirming the profile

Landing Page

I used the "Import Site" feature to clone a login page. Gophish fetches the HTML of any URL you give it and turns it into a landing page. I enabled credential capture so that any username/password submitted gets logged.

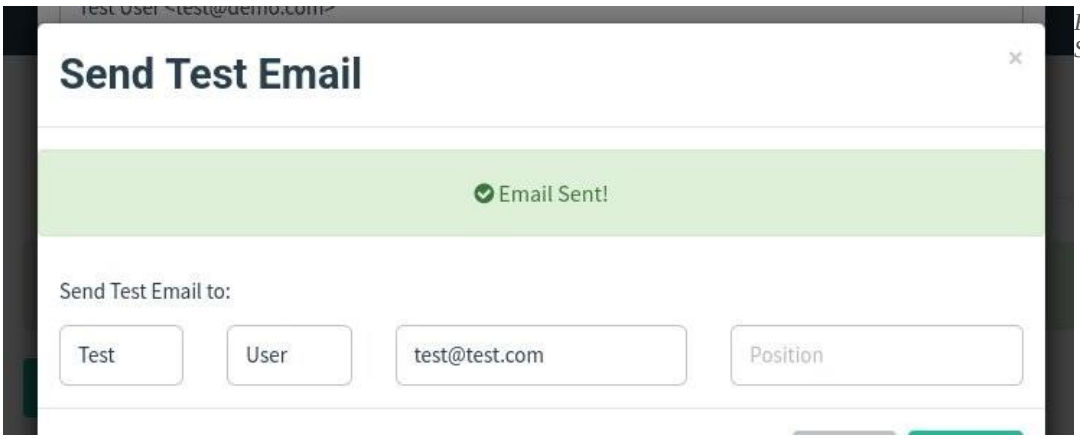


Figure 10 – Import Site dialog

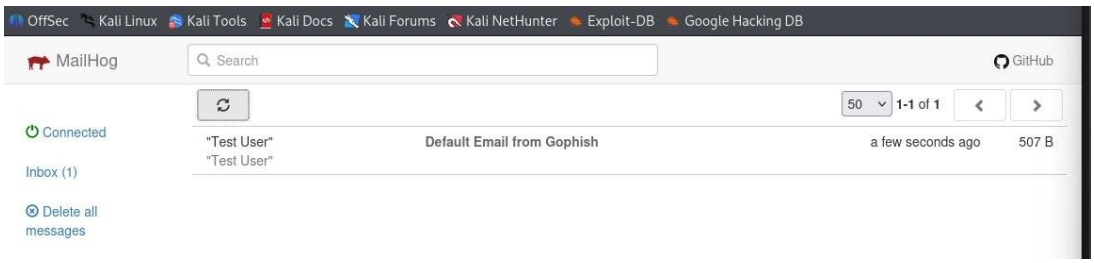


Figure 11 – Landing Pages list showing "Login Page" saved

Email Template

The email template was called "Password Reset". The subject line was "Urgent: Reset Your Password" to create a sense of urgency. The body told the target that unusual activity had been detected and they needed to click the reset link immediately. The link uses Gophish's {{.URL}} variable which gets replaced with a unique tracked URL per recipient.

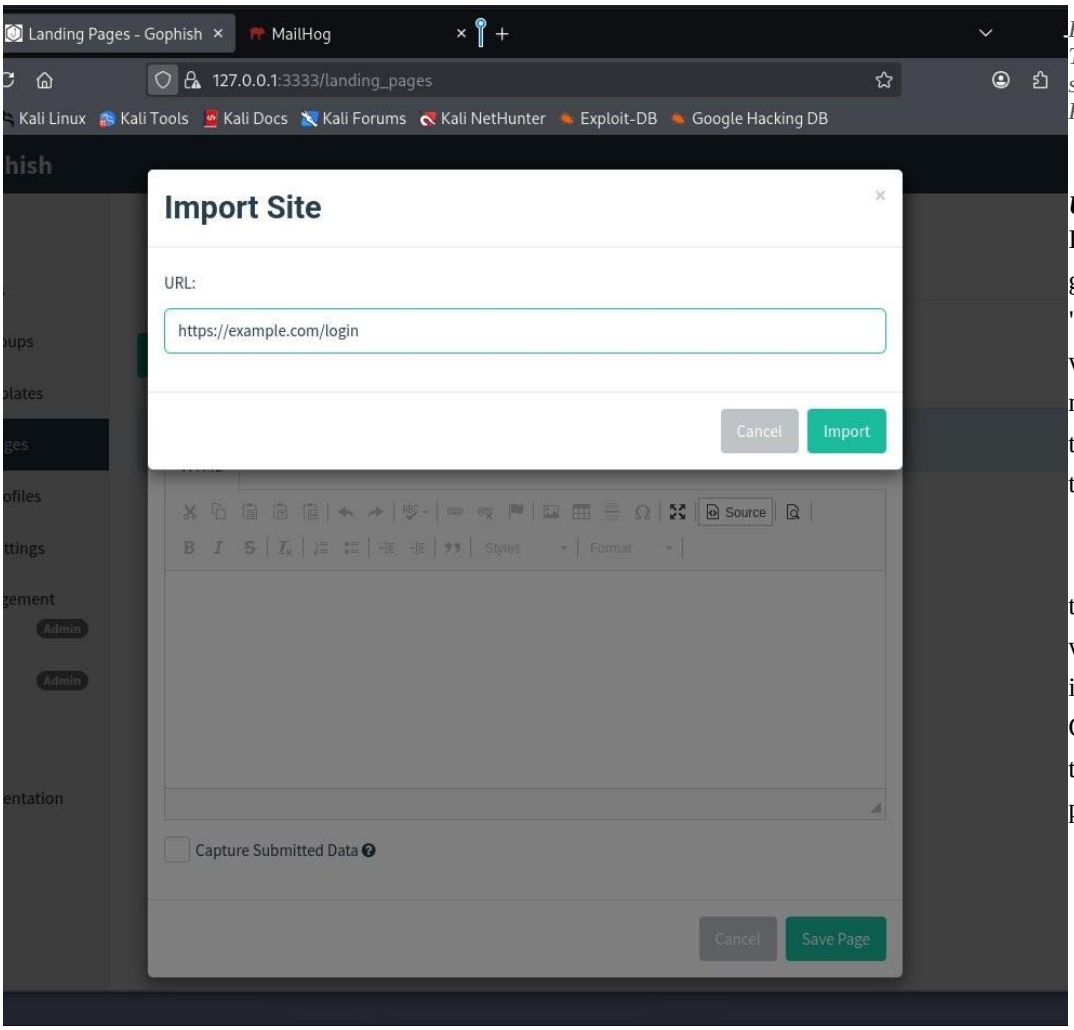


Figure 12 – Email Templates list showing "Password Reset"

Users and Groups

I created a group called "Test Group" with one member — the synthetic test user. In a real engagement this group would be imported as a CSV from the OSINT phase.

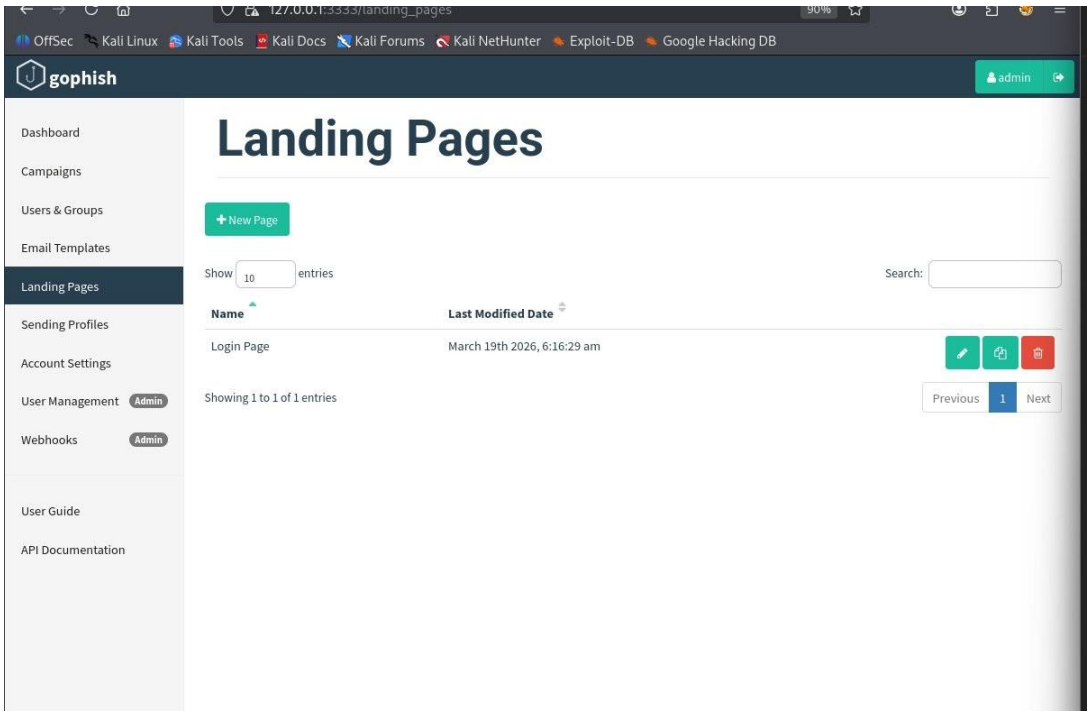


Figure 13 – Users & Groups showing Test Group with 1 member

4.4 Phase 4 – Campaign Execution and Results

Test email validation

Before launching the full campaign I sent a test email from the sending profile configuration page to make sure delivery was working. MailHog received it straight away.

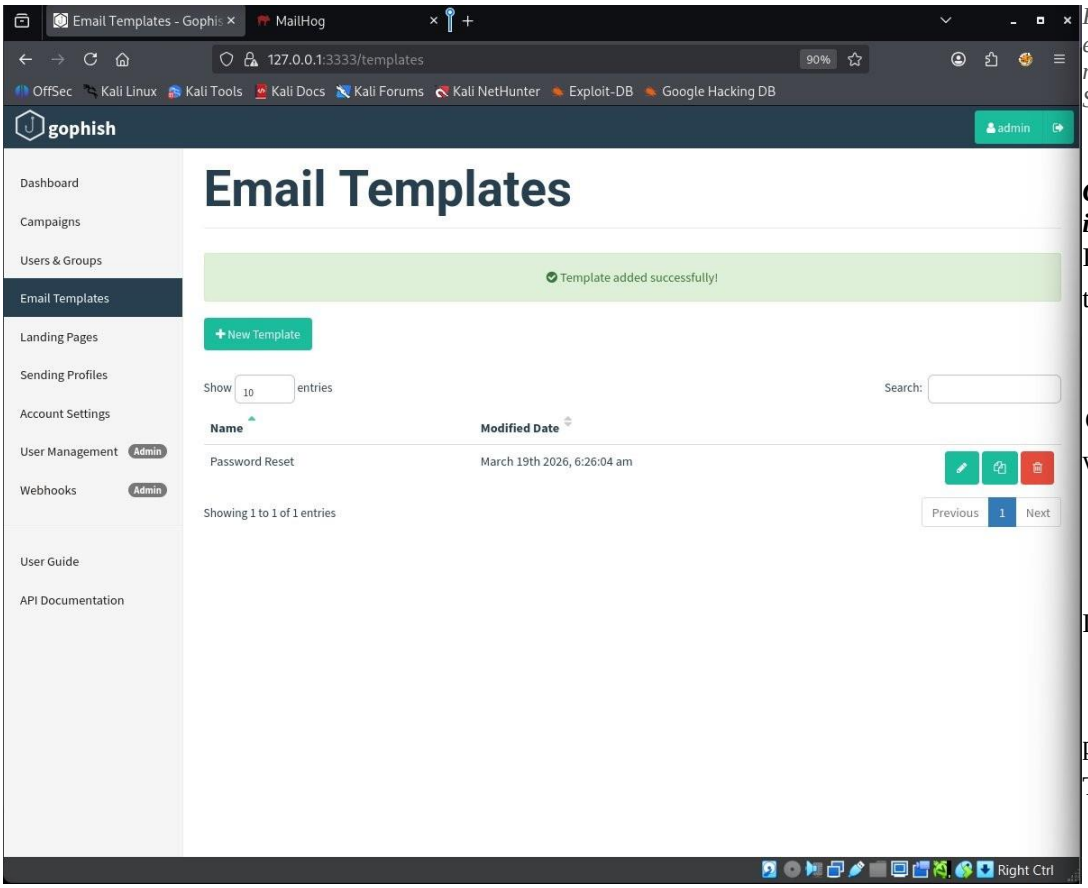


Figure 14 – Test email confirmation modal (Email Sent!)

Campaign 1 – initial launch

I launched the first campaign ("Test Campaign") with the Password Reset template, Login Page, MailHog sending profile, and Test Group. The campaign was scheduled and the email went out immediately.

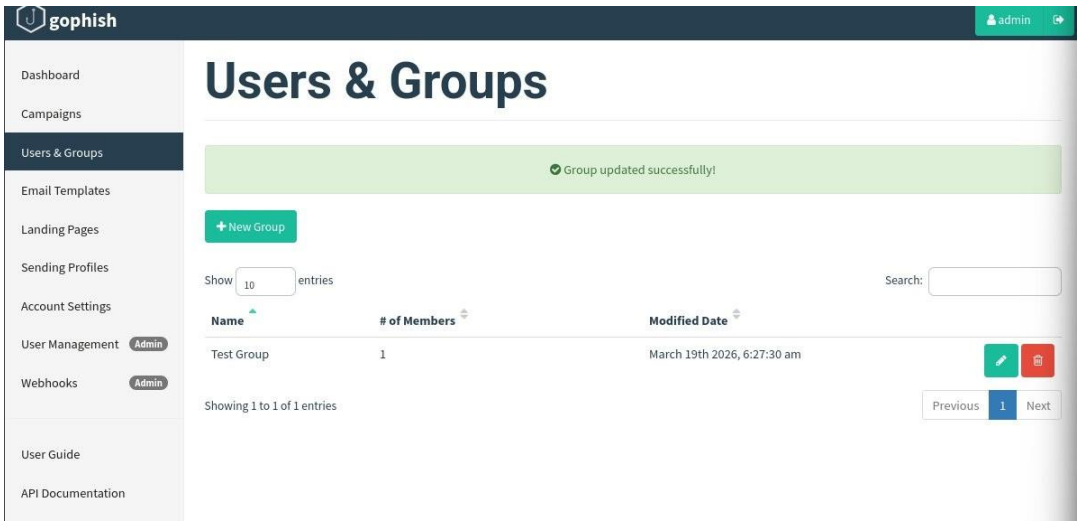


Figure 15 – New Campaign configuration form

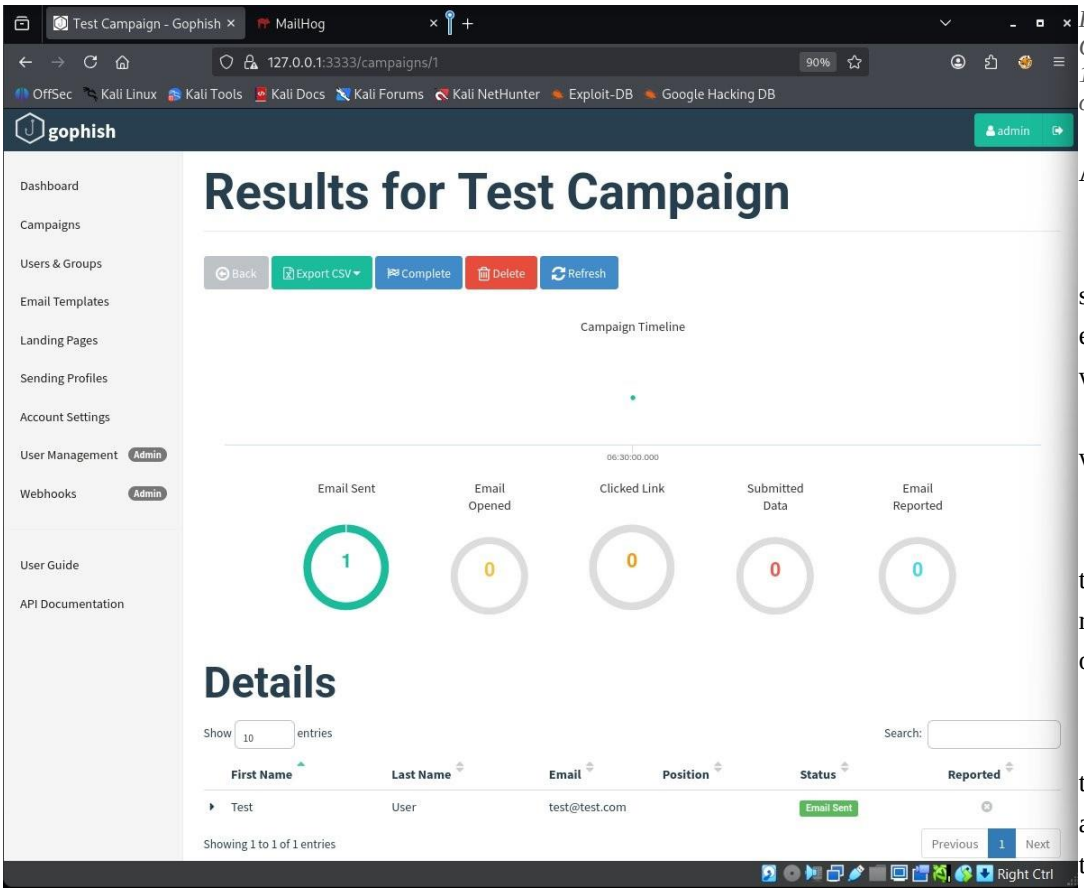


Figure 17 – Campaign results: 1 sent, 0 opened/clicked

At this stage the campaign showed 1 email sent with zero interaction, which is expected because there is no real user to open it. MailHog confirmed the email arrived in the inbox.

Campaign 3 – full simulation with interaction

For the second run I configured another campaign (Campaign 3) with the same settings. This time I acted as the target myself to simulate what would happen if a real employee received and interacted with the phishing email.

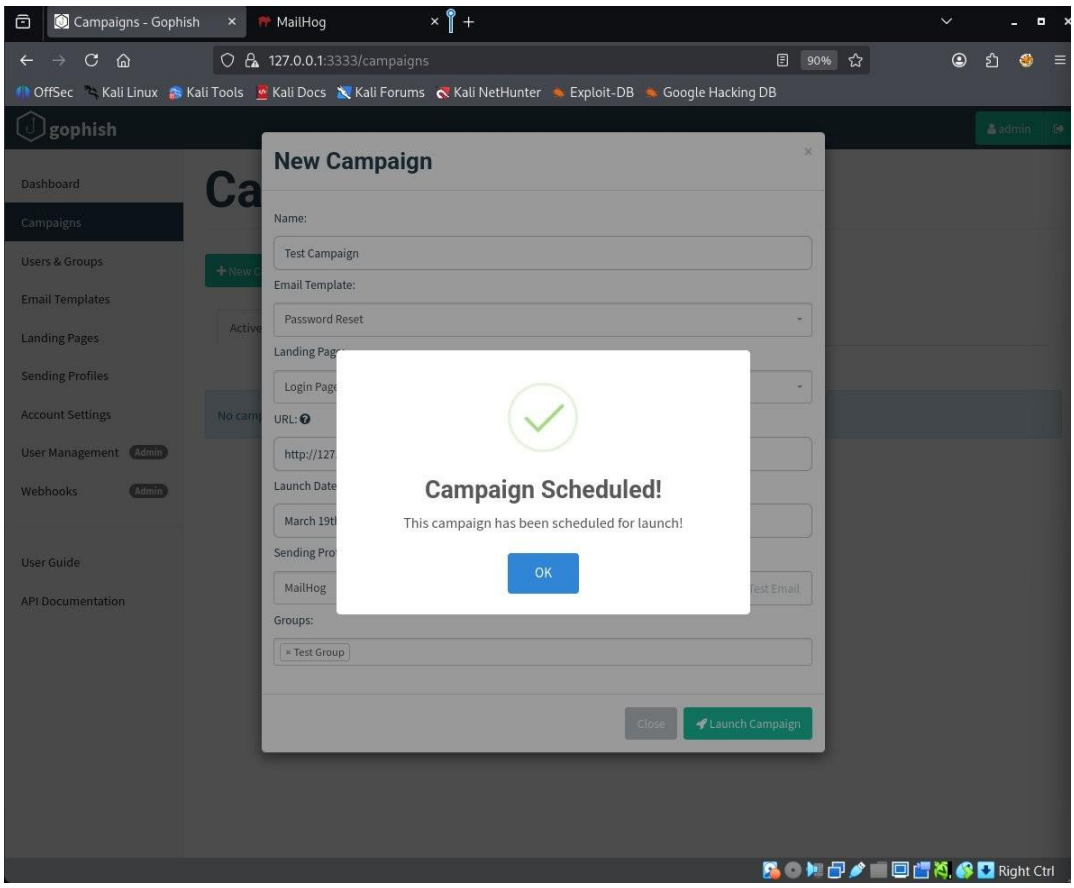


Figure 20 – Campaign Scheduled confirmation

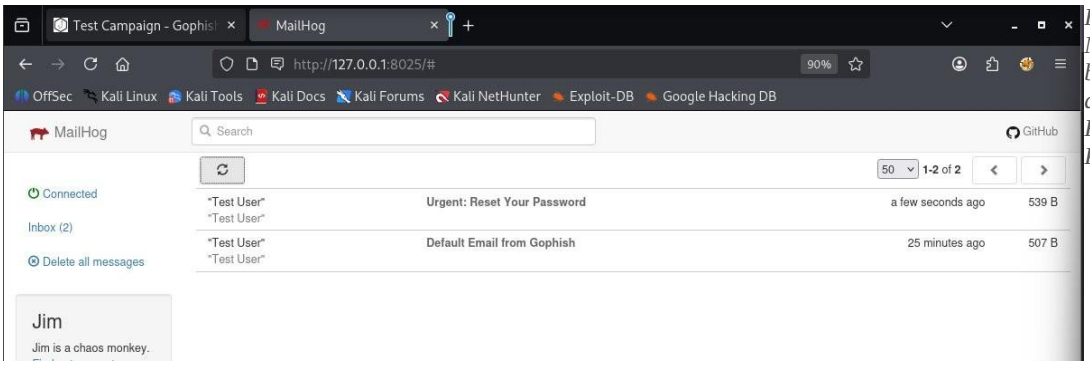


Figure 18 – MailHog inbox: both the test email and the "Urgent: Reset Your Password" email

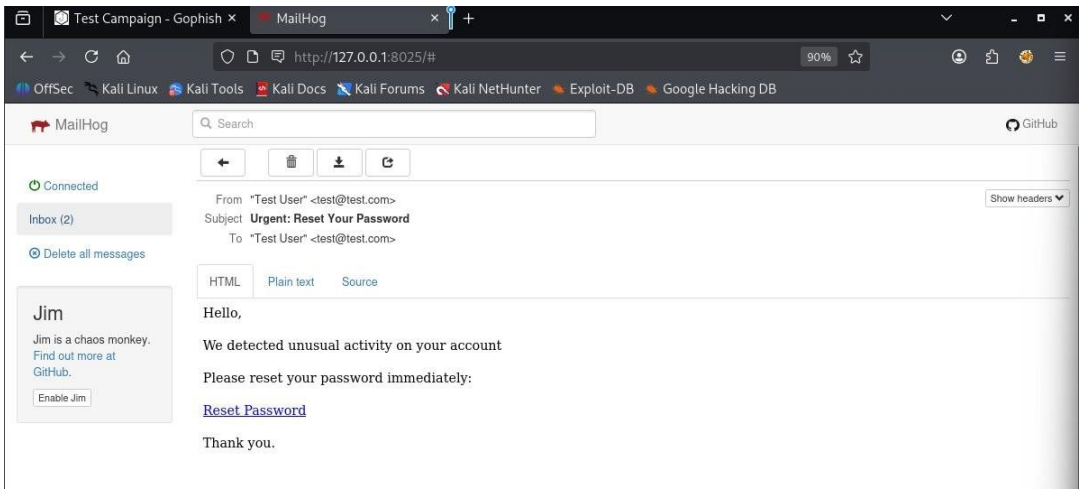


Figure 19 – Email content in MailHog showing the phishing message and "Reset Password" link

I opened the email in MailHog, which triggered the Email Opened event. Then I clicked the "Reset Password" link, which triggered Clicked Link. The link opened the fake login page at http://127.0.0.1/?rid=g3ANPfB:

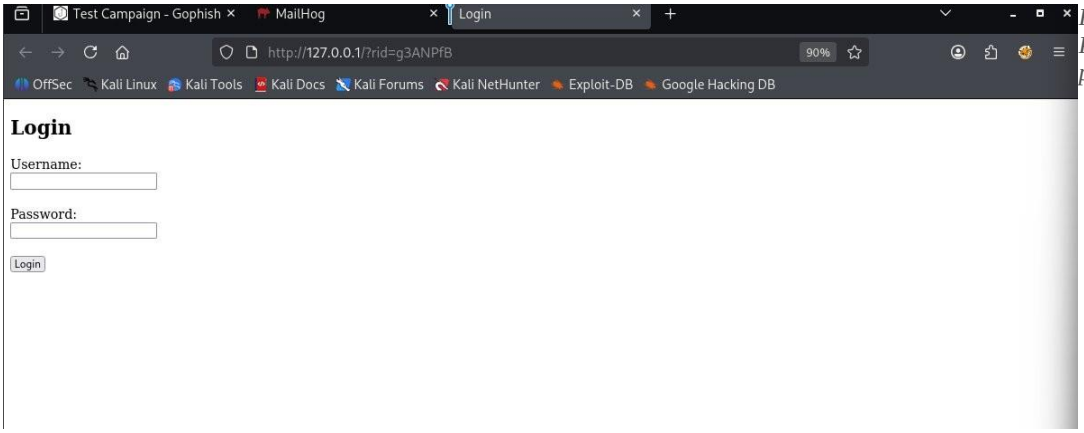


Figure 21 – Phishing landing page at http://127.0.0.1/?rid=g3ANPfB

I entered test credentials on the login form and clicked "Login". This triggered the Submitted Data event on the Gophish dashboard. The final campaign results are shown below:

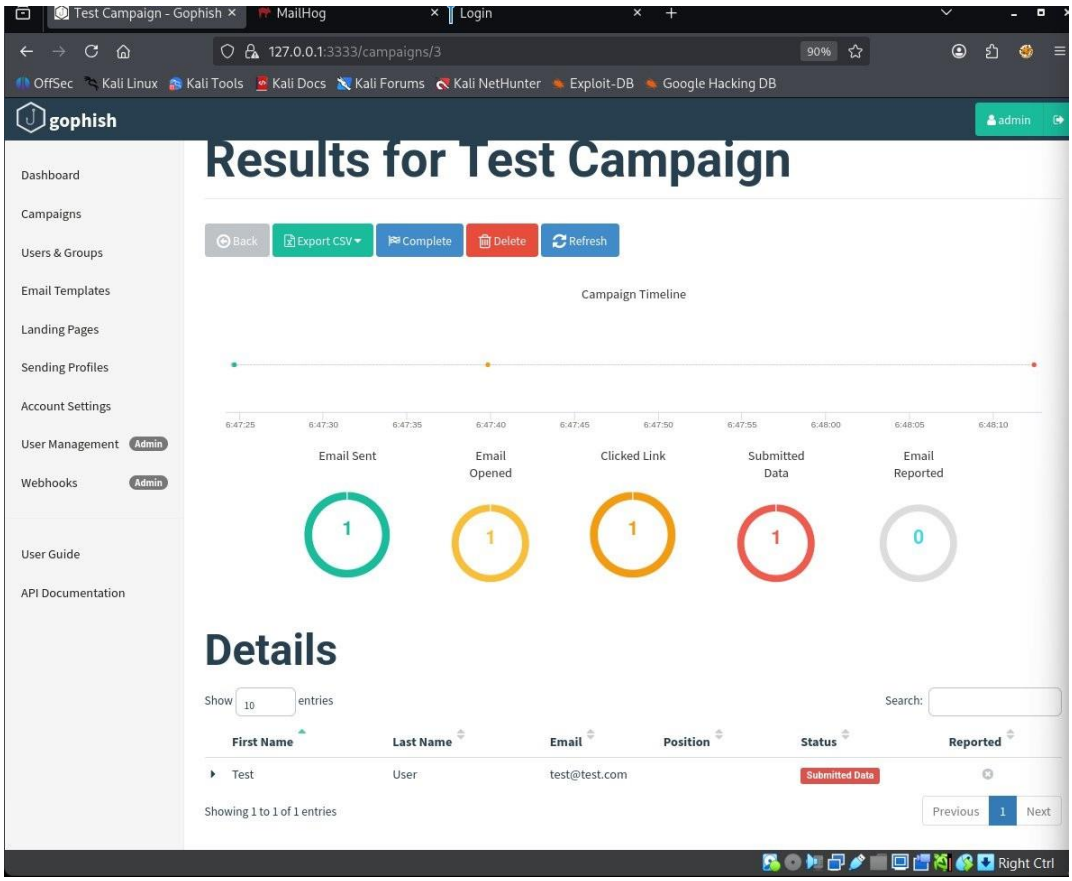


Figure 22 – Final campaign results: Sent 1, Opened 1, Clicked 1, Submitted Data 1

Metric	Result	What this means
Email Sent	1 / 1 (100%)	Email delivered successfully
Email Opened	1 / 1 (100%)	Tracking pixel loaded; target opened the email
Clicked Link	1 / 1 (100%)	Target clicked the phishing URL
Submitted Data	1 / 1 (100%)	Target entered credentials on the fake login page
Email Reported	0 / 1 (0%)	Target did not report the email as phishing

The campaign timeline showed all four events happening within about 90 seconds (06:47:25 to 06:48:10), which is realistic — users who fall for phishing usually act on it quickly.

5. Findings and Evidence

Based on the simulation results, here are the main findings. I have rated each one by how serious it would be in a real environment.

Finding 1 – Severity: High
Phishing infrastructure can be set up in under 10 minutes
Gophish and MailHog were both running and working in less than 10 minutes. The tools are free, well-documented, and easy to use. This shows how little effort it takes for an attacker to build a functional phishing operation. Evidence: Figures 1–5.

Finding 2 – Severity: High
Login page cloned in one click with no technical skill required
The Import Site feature in Gophish clones any login page with a single URL. The resulting page is visually identical to the real one. A victim would have no obvious visual cue that it is fake. Evidence: Figures 10–11, 21.

Finding 3 – Severity: High
Phishing email with spoofed sender delivered without any filtering
The email used a spoofed From address and was delivered straight to the inbox with no warnings. In a real environment without SPF, DKIM, and DMARC, this would reach the target just as easily. Evidence: Figures 18–19.

Finding 4 – Severity: High
Credentials submitted on the fake login page (100% success rate)
In Campaign 3, the target went through the full phishing flow: email opened, link clicked, credentials entered. In a real attack these credentials would now be in the attacker's hands. Evidence: Figure 22.

Finding 5 – Severity: Medium
No MFA on the Gophish admin panel
The admin panel was protected by username and password only. If an attacker got hold of these credentials, they could see all campaign data, harvested credentials, and target information.

Finding 6 – Severity: Medium
Phishing URL served over HTTP (no HTTPS)
The landing page ran on plain HTTP. Modern browsers may show a "Not Secure" warning, but many users ignore or do not notice these. In a real attack the page would likely use HTTPS with a free certificate to look more legitimate.

Finding 7 – Severity: Low
No phishing report made by the target
The Email Reported count stayed at 0. In a real scenario this means the security team would not have been alerted by the user, so detection would have to come from technical controls rather than user reporting.

6. Mitigation Strategies

These are the defences that would have the most impact against the kind of phishing attack demonstrated in this simulation.

Technical Controls

Set up SPF, DKIM, and DMARC
SPF tells receiving mail servers which servers are allowed to send email for your domain. DKIM adds a cryptographic signature so the email cannot be tampered with in transit. DMARC builds on both and lets you tell mail servers to reject or quarantine emails that fail those checks. Together these make it very hard to spoof your domain. DMARC record example: v=DMARC1; p=reject; rua=mailto:dmarc@company.com

Enforce multi-factor authentication (MFA) on all accounts
Even if a user submits their password on a phishing page, MFA means the attacker still cannot log in. This is one of the highest-value controls you can put in place. Hardware keys (FIDO2/WebAuthn) are the best option for high-risk accounts.

Deploy an email security gateway
Tools like Microsoft Defender for Office 365, Proofpoint, or Mimecast analyse inbound email for phishing indicators, rewrite URLs to scan them at click time, and sandbox attachments before delivery.

DNS and web filtering
Block access to known phishing domains and newly registered domains at the DNS or web proxy level. This would stop the user reaching the phishing page even if they clicked the link.

Browser security settings
Enable Google Safe Browsing or equivalent. Modern browsers warn users about known phishing sites — keeping browsers updated ensures these protections are current.

Procedural and Training Controls

Regular phishing awareness training
Run training sessions at least twice a year covering how to spot phishing emails: suspicious sender addresses, urgency language, hovering over links before clicking, and the importance of reporting suspicious emails.

Phishing simulation programme
Run regular simulated phishing campaigns (quarterly or monthly) and use the results to identify which users need extra training. Track click rates over time to see whether awareness is improving.

Clear incident reporting process
Make it easy and safe for users to report phishing. A "Report Phishing" button in the email client removes friction. If users fear being blamed, they will not report.

Principle of least privilege

Limit what each account can access. If a phished user only has access to what they need for their job, the blast radius of a successful attack is much smaller.

7. Incident Response Simulation

This section walks through how a security team would respond if this phishing attack happened in a real organisation. I have followed the NIST SP 800-61 Rev. 2 framework: Preparation Detection Containment Eradication Recovery Post-Incident Review.

7.1 Detecting the Attack

In this simulation the detection indicators would be:

– Gophish logs showing email_sent, email_opened, clicked_link, and submitted_data events all for the same target within 90 seconds.

– The mail gateway logging an inbound email with a From address that does not match the SPF record for the sending domain.

– Proxy or DNS logs showing a user visiting a newly registered or unknown domain after clicking a link in an email.

– SIEM alert triggered by a POST request to a login form from an IP not associated with normal user activity.

– Credential submitted to an unknown endpoint — detectable by user behaviour analytics (UEBA) tools.

Gophish log evidence from this simulation:

```
Campaign: Test Campaign (Campaign 3) 06:47:25 - Email Sent  test@test.com
06:47:30 - Email Opened  IP: 127.0.0.1 User-Agent: Firefox/Kali 06:47:45 -
Clicked Link  http://127.0.0.1/?rid=g3ANPfB 06:47:55 - Submitted Data 
Credentials captured on login page
```

7.2 Im

– Disable or lock the affected user account.

– Block the phishing URL at the web proxy and DNS firewall.

– Send an alert to all staff warning them about the campaign.

– Notify the email security team to quarantine similar emails still in transit.

Short-term – within 4 hours

– Force a password reset for the user who submitted credentials.

– Revoke all active sessions and OAuth tokens for that account.

– Search for and delete the phishing email from all mailboxes using admin tools.

– Request takedown of the phishing domain from the registrar and hosting provider.

Recovery – within 48 hours

– Restore account access after password reset and MFA enrolment.

– Monitor the account for unusual login activity for at least 30 days.

– Scan the affected workstation for any malware dropped during the session.

– Update threat intelligence feeds with the phishing domain and IP as indicators of compromise.

7.3 Post-Incident Report

A post-incident report is written after the response is complete. It documents what happened, what the impact was, why existing controls did not stop it, and what changes need to be made. The key sections are:

Section	Content
Incident Summary	What happened, who was targeted, and when.
Timeline	Chronological log of all events.
Impact	Accounts affected, data exposed, business disruption.
Root Cause	Why controls failed (e.g. no DMARC, no MFA).
Containment Actions	Steps taken to stop the attack spreading.
Remediation	Password resets, account audits, infrastructure takedowns.
Lessons Learned	What needs to change and why.
Follow-up Tasks	Owners and deadlines for each action.

8. Conclusion

This project gave me hands-on experience with the full phishing attack lifecycle. I went from downloading the tools to having a working phishing campaign that could deliver an email, get the target to open it, click the link, and submit their credentials on a fake login page — all within a safe, isolated lab.

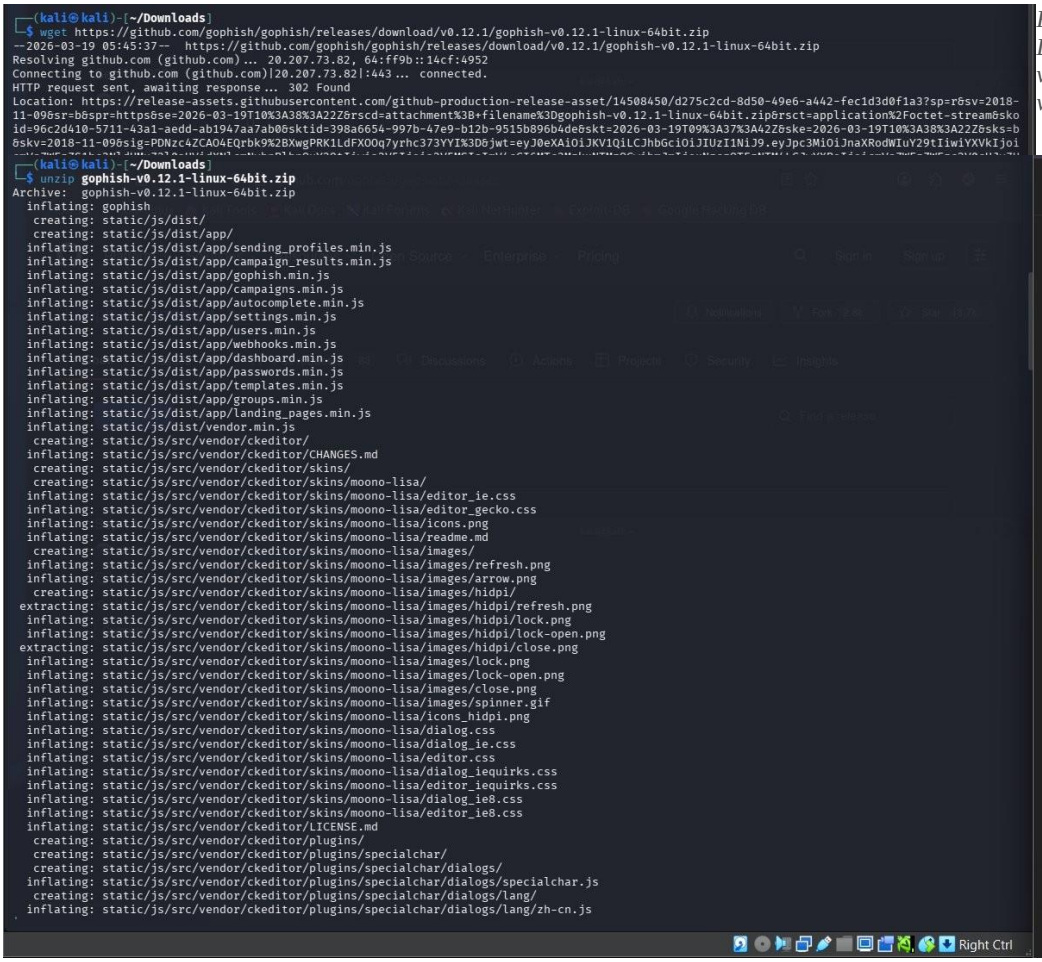
The most important takeaway is how easy it is to run a convincing phishing campaign. The tools are free and well-documented, the email was delivered without any authentication checks blocking it, and the landing page looked like a real login form. In a real organisation, any one of several defences — DMARC, MFA, email filtering, user awareness — could have broken the attack chain. The problem is that these controls need to all be in place together; a gap in any one of them creates an opening.

From a defensive perspective, the most high-value steps an organisation can take are: enforcing MFA on all accounts, implementing DMARC with a reject policy, deploying an email security gateway, and running regular phishing simulations so staff learn to recognise and report suspicious emails.

Prepared by: Intern – Apex Planet Cybersecurity **Date:** March 21, 2026

9. Appendix – All Screenshots

All screenshots taken during the exercise are included below in chronological order.



```
┌──(kali㉿kali)-[~/Downloads]
└─$ wget https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-v0.12.1-linux-64bit.zip
--2026-03-19 05:45:37--  https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-v0.12.1-linux-64bit.zip
Resolving github.com (github.com)... 20.207.73.82, 64:ff9b::14cf:4952
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/14508450/d275c2cd-8d50-49e6-a442-fec1d3d0f1a3?sp=r&sv=2018-
11-09&sr=b&spr=https&se=2026-03-19T10%3A38%3A22Z&rscd=attachment%3B+filename%3Dgophish-v0.12.1-linux-64bit.zip&rsct=application%2Foctet-stream&sko
id=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-03-19T09%3A37%3A42Z&ske=2026-03-19T10%3A38%3A22Z&sks=b
&skv=2018-11-09&sig=PDNzc4ZCAO4EQrbk9%2BXwgPRK1LdFXOOq7yrhc373YYI%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoi
```

Figure 1 – Downloading Gophish v0.12.1 binary using wget

```
┌──(kali㉿kali)-[~/Downloads]
└─$ unzip gophish-v0.12.1-linux-64bit.zip
Archive:  gophish-v0.12.1-linux-64bit.zip
  inflating: gophish
   creating: static/js/dist/
   creating: static/js/dist/app/
  inflating: static/js/dist/app/sending_profiles.min.js
  inflating: static/js/dist/app/campaign_results.min.js
  inflating: static/js/dist/app/gophish.min.js
  inflating: static/js/dist/app/campaigns.min.js
  inflating: static/js/dist/app/autocomplete.min.js
  inflating: static/js/dist/app/settings.min.js
  inflating: static/js/dist/app/users.min.js
  inflating: static/js/dist/app/webhooks.min.js
  inflating: static/js/dist/app/dashboard.min.js
  inflating: static/js/dist/app/passwords.min.js
  inflating: static/js/dist/app/templates.min.js
  inflating: static/js/dist/app/groups.min.js
  inflating: static/js/dist/app/landing_pages.min.js
  inflating: static/js/dist/vendor.min.js
   creating: static/js/src/vendor/ckeditor/
  inflating: static/js/src/vendor/ckeditor/CHANGES.md
   creating: static/js/src/vendor/ckeditor/skins/
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_ie.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_gecko.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/icons.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/readme.md
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/refresh.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/arrow.png
   creating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/
 extracting: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/refresh.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/lock.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/lock-open.png
 extracting: static/js/src/vendor/ckeditor/skins/moono-lisa/images/hidpi/close.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/lock.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/lock-open.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/close.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/images/spinner.gif
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/icons_hidpi.png
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_ie.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_iequirks.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_iequirks.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/dialog_ie8.css
  inflating: static/js/src/vendor/ckeditor/skins/moono-lisa/editor_ie8.css
  inflating: static/js/src/vendor/ckeditor/LICENSE.md
   creating: static/js/src/vendor/ckeditor/plugins/
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/
  inflating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/specialchar.js
   creating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/lang/
  inflating: static/js/src/vendor/ckeditor/plugins/specialchar/dialogs/lang/zh-cn.js
```

Figure 2 – Extracting the Gophish archive

```
┌──(kali㉿kali)-[~/Downloads]
└─$ chmod +x gophish

┌──(kali㉿kali)-[~/Downloads]
└─$ ls -la
total 190660
drwxr-xr-x  5 kali kali     4096 Mar 19 05:47 .
drwx------ 30 kali kali     4096 Mar 19 05:45 ..
-rw-rw-r--  1 kali kali      940 Mar 15 09:17 cacert.der
-rw-r--r--  1 kali kali      460 Sep 14  2022 config.json
drwxr-xr-x  4 kali kali     4096 Sep 14  2022 db
-rwxr-xr-x  1 kali kali 21525728 Sep 14  2022 gophish
-rw-rw-r--  1 kali kali 33327057 Sep 14  2022 gophish-v0.12.1-linux-64bit.zip
-rw-r--r--  1 kali kali     1115 Sep 14  2022 LICENSE
-rw-rw-r--  1 kali kali 69359018 Mar 10 03:19 Nessus-10.11.3-debian10_amd64.deb
-rw-rw-r--  1 kali kali 70971470 Mar 10 08:08 Nessus-10.11.3-ubuntu1604_amd64.deb
-rw-r--r--  1 kali kali     3285 Sep 14  2022 README.md
drwxrwxr-x  7 kali kali     4096 Mar 19 05:47 static
drwxr-xr-x  2 kali kali     4096 Sep 14  2022 templates
-rw-r--r--  1 kali kali        7 Sep 14  2022 VERSION

┌──(kali㉿kali)-[~/Downloads]
└─$ sudo ./gophish
time="2026-03-19T05:50:11-04:00" level=warning msg="No contact address has been configured."
time="2026-03-19T05:50:11-04:00" level=warning msg="Please consider adding a contact_address entry in your config.json"
goose: migrating db environment 'production', current version: 0, target: 20220321133237
OK    20160118194630_init.sql
OK    20160131153104_0.1.2_add_event_details.sql
OK    20160211211220_0.1.2_add_ignore_cert_errors.sql
OK    20160217211342_0.1.2_create_from_col_results.sql
OK    20160225173824_0.1.2_capture_credentials.sql
OK    20160227180335_0.1.2_store-smtp-settings.sql
OK    20160317214457_0.2_redirect_url.sql
OK    20160605210903_0.2_campaign_scheduling.sql
OK    20170104220731_0.2_result_statuses.sql
OK    20170219122503_0.2.1_email_headers.sql
```

Figure 3 – Making the binary executable and running Gophish for the first time

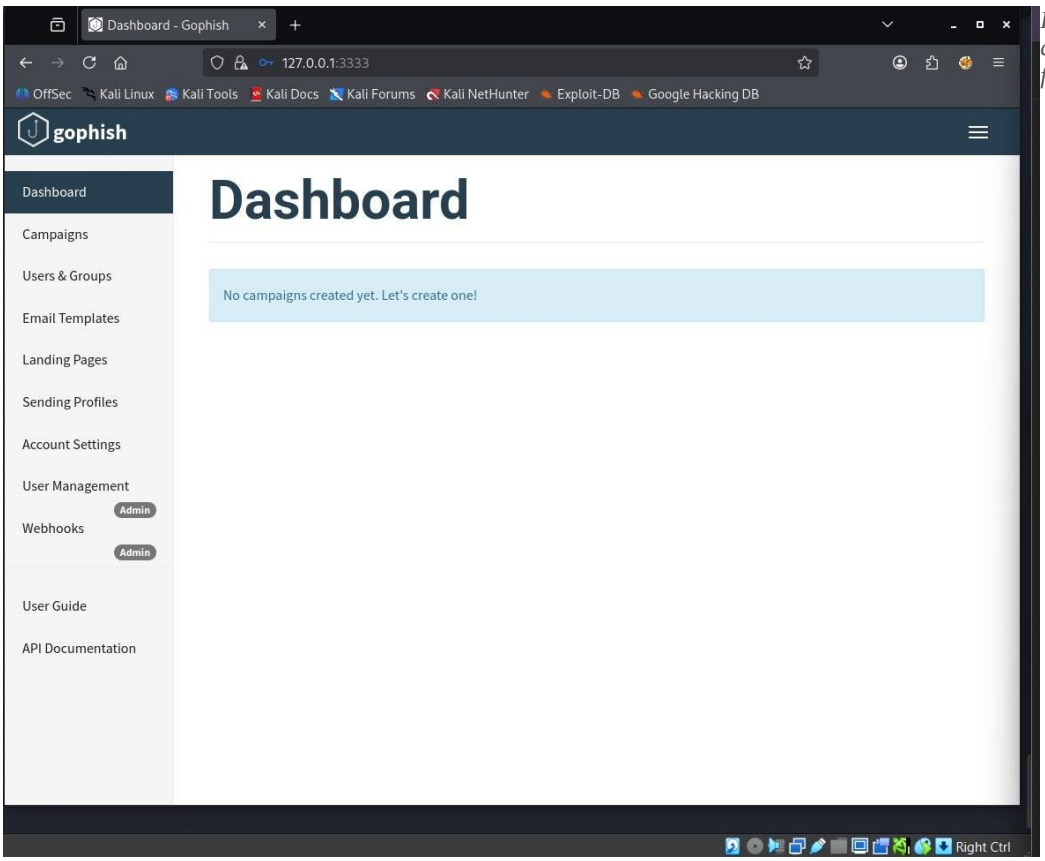


Figure 4 – Gophish admin dashboard after first login

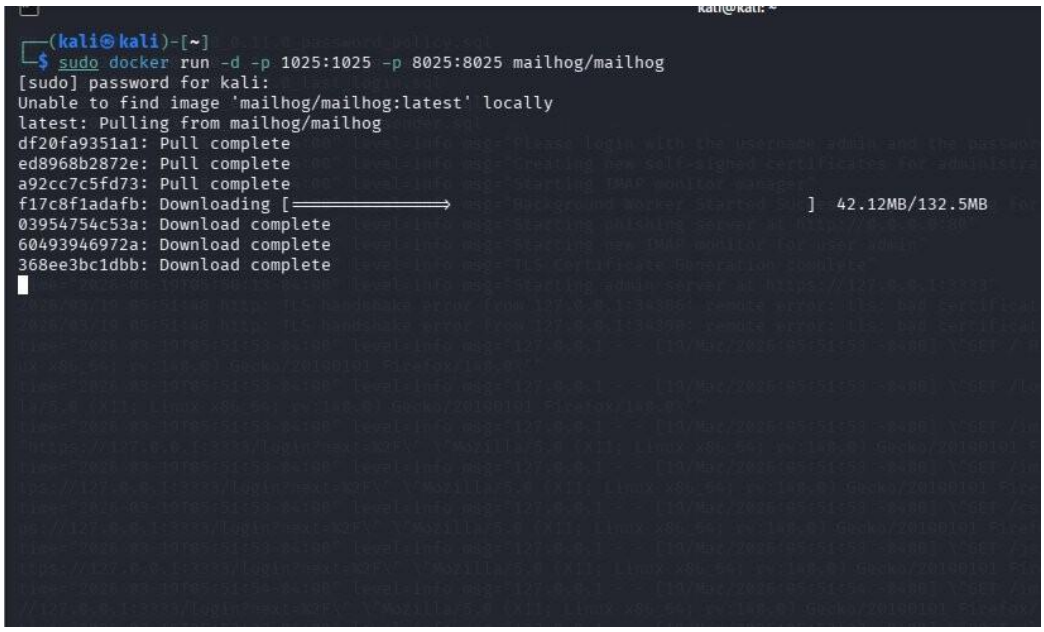


Figure 5 – MailHog web interface (empty inbox on startup)

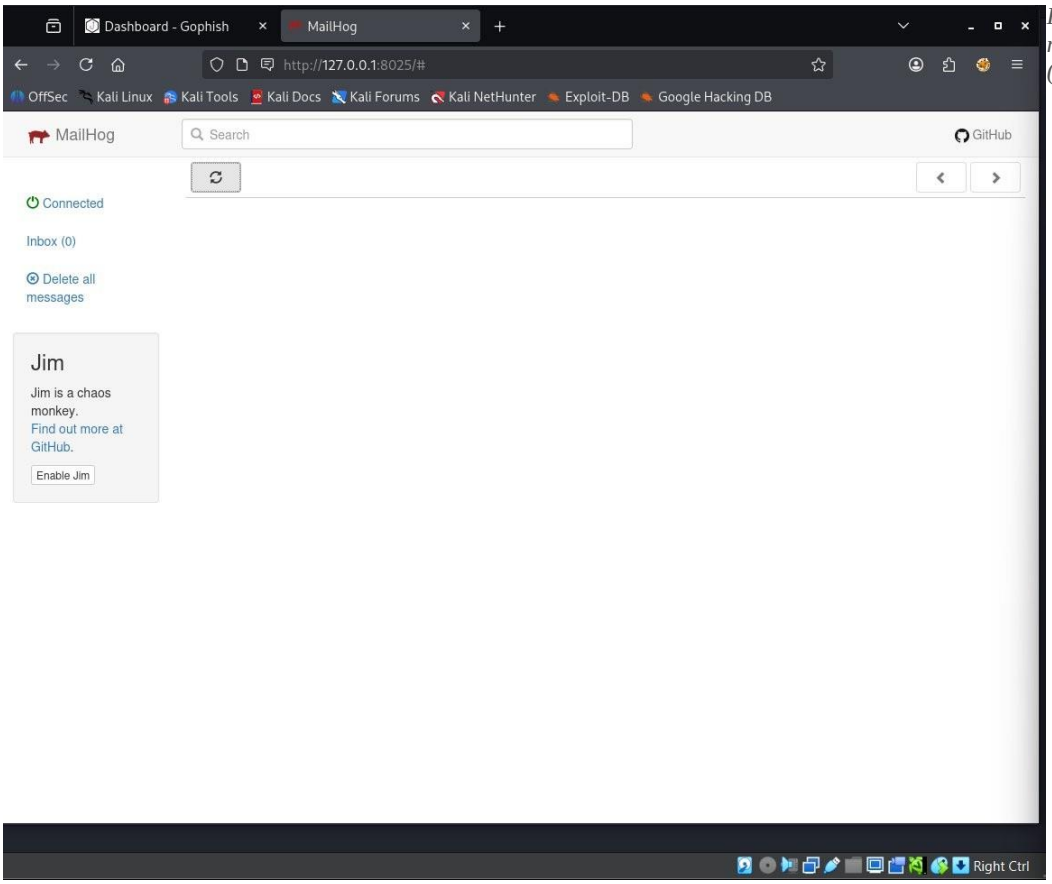


Figure 6 – Creating a new Sending Profile (MailHog SMTP)

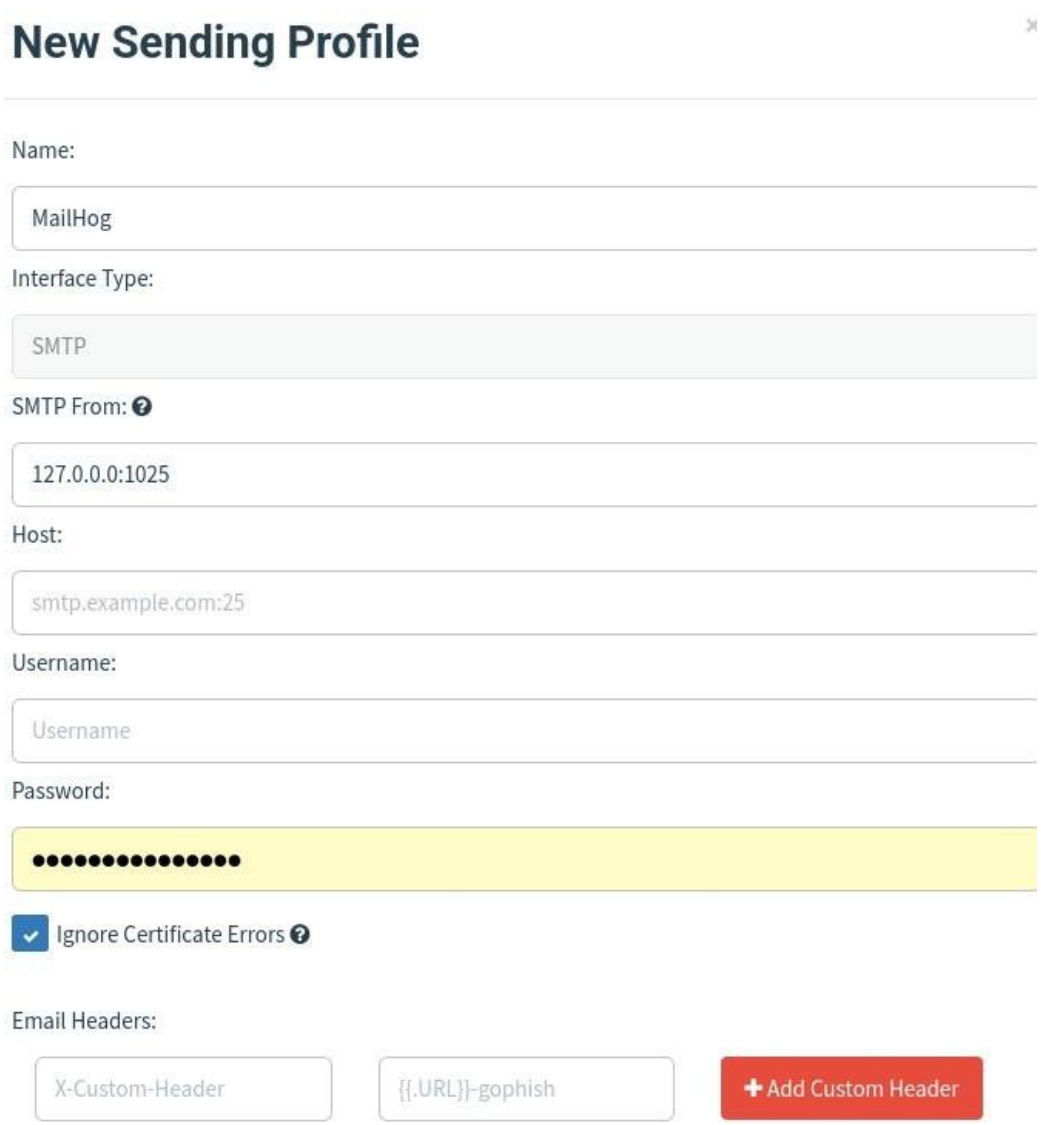


Figure 7 – Sending Profile form – additional fields

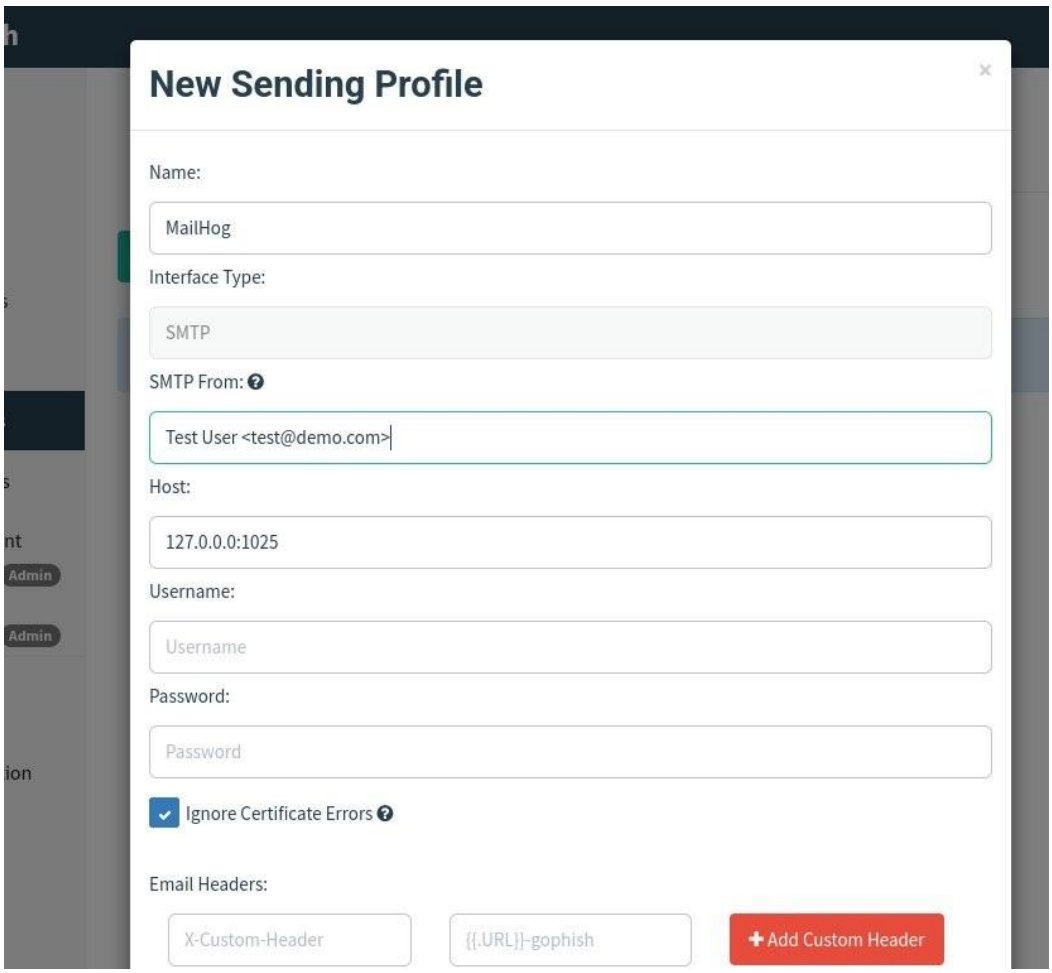


Figure 8 – Sending Profile saved successfully

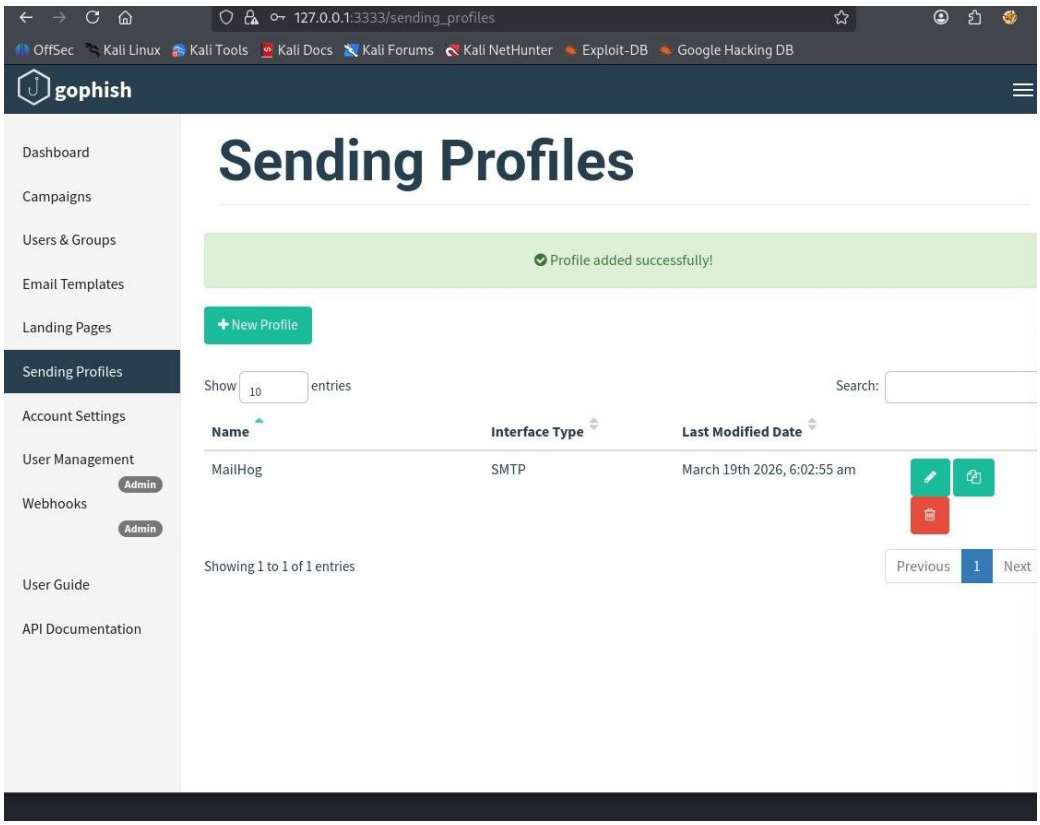


Figure 9 – Sending Profiles list confirming MailHog profile

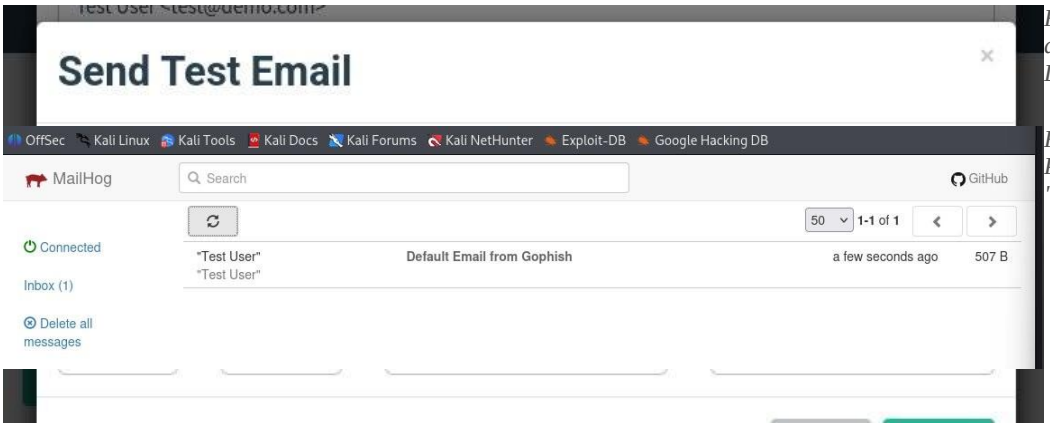


Figure 10 – Import Site dialog for creating a Landing Page

Figure 11 – Landing Pages list showing "Login Page" created

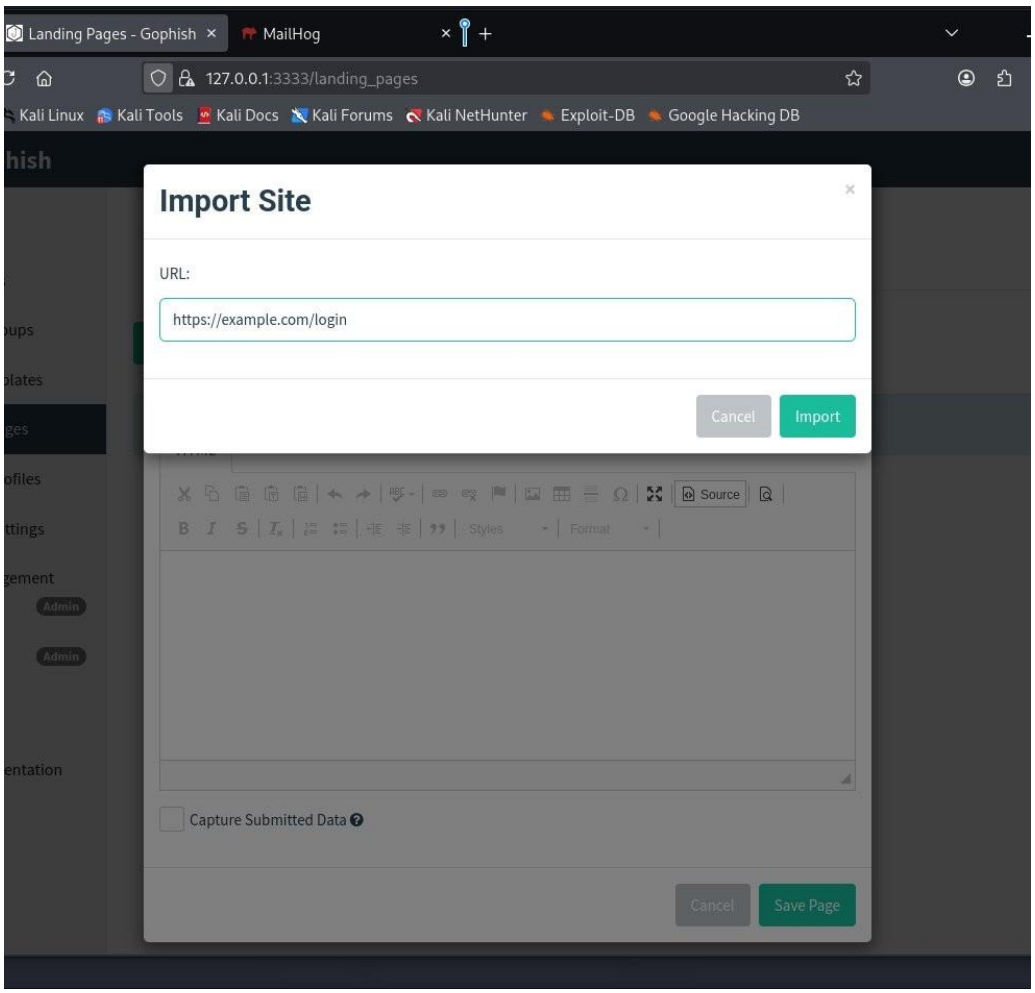


Figure 12 – Email Templates – "Password Reset" template added

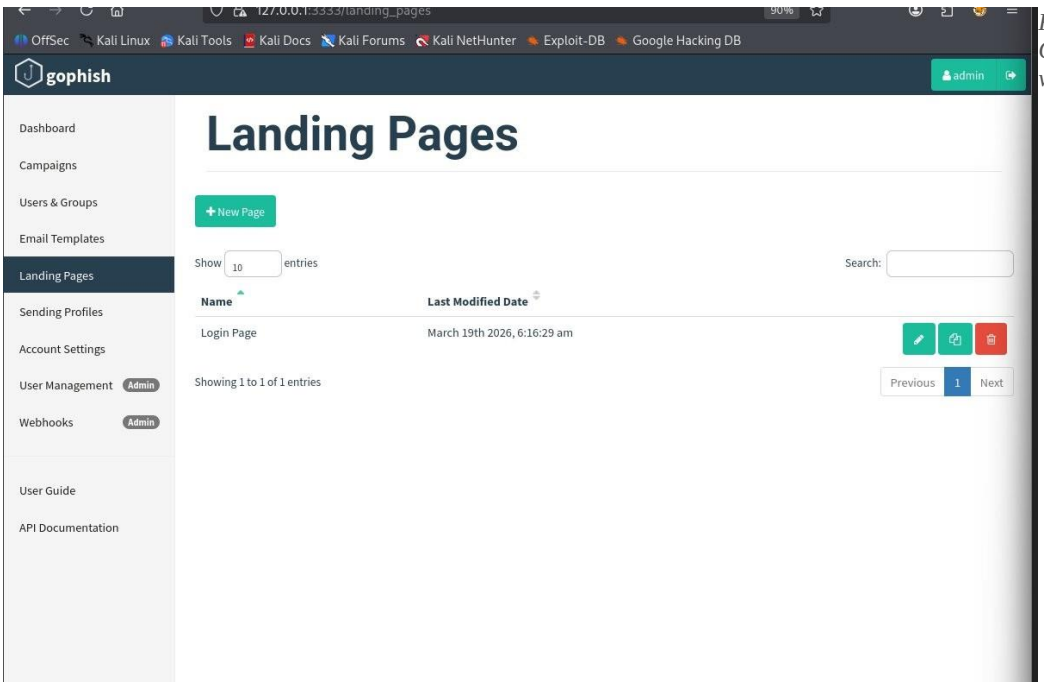


Figure 13 – Users & Groups – Test Group with one member saved

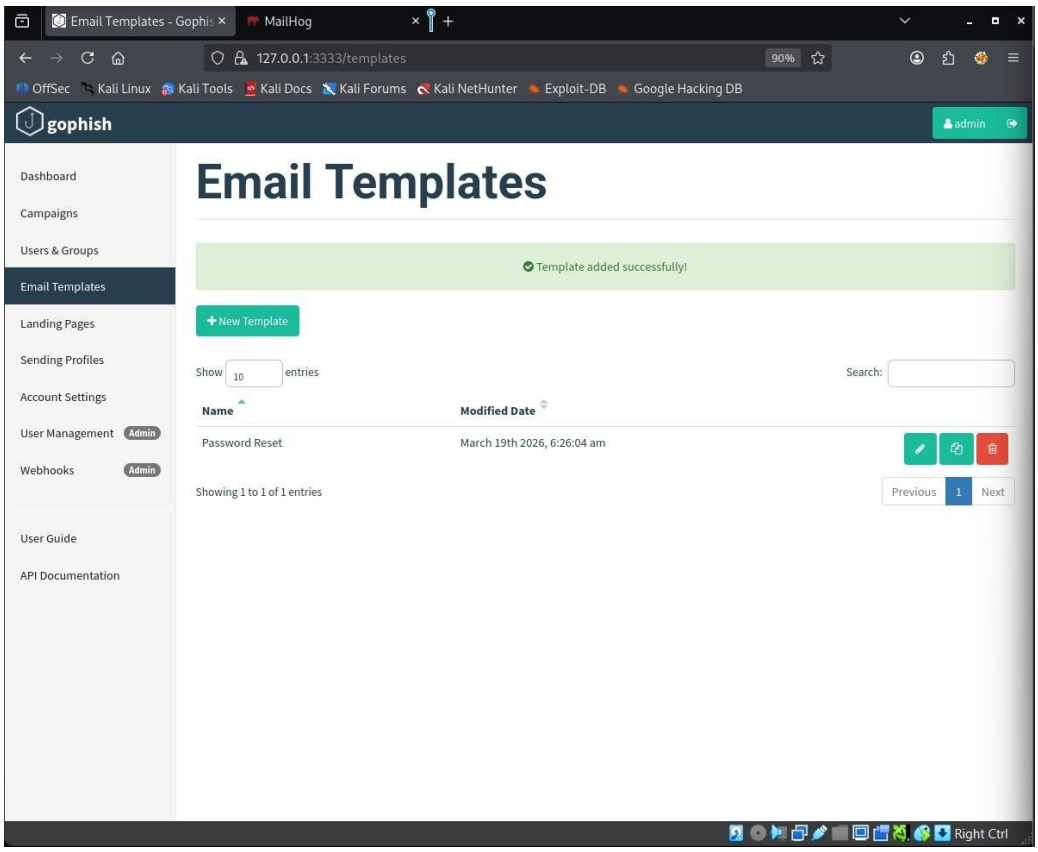


Figure 14 – Test email sent and confirmed (Email Sent! modal)

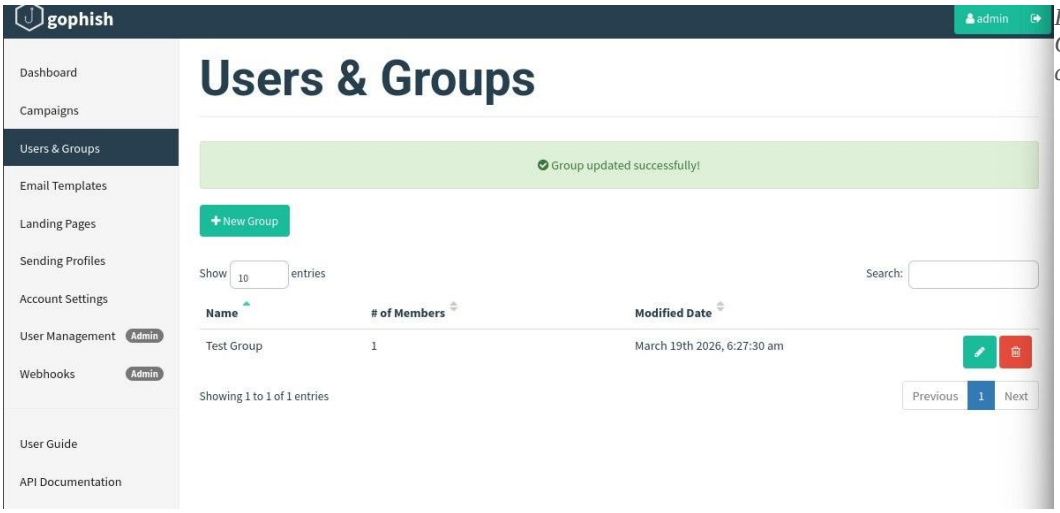


Figure 15 – New Campaign configuration form

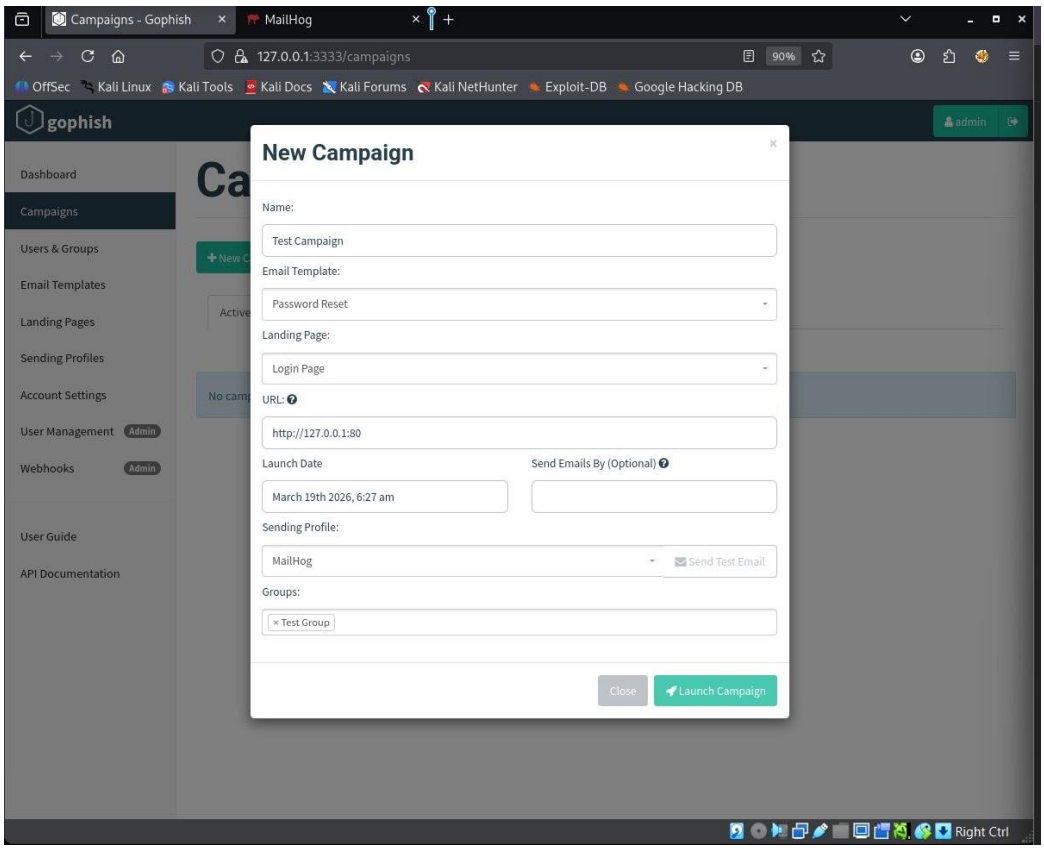


Figure 16 – MailHog inbox showing the first Gophish email received

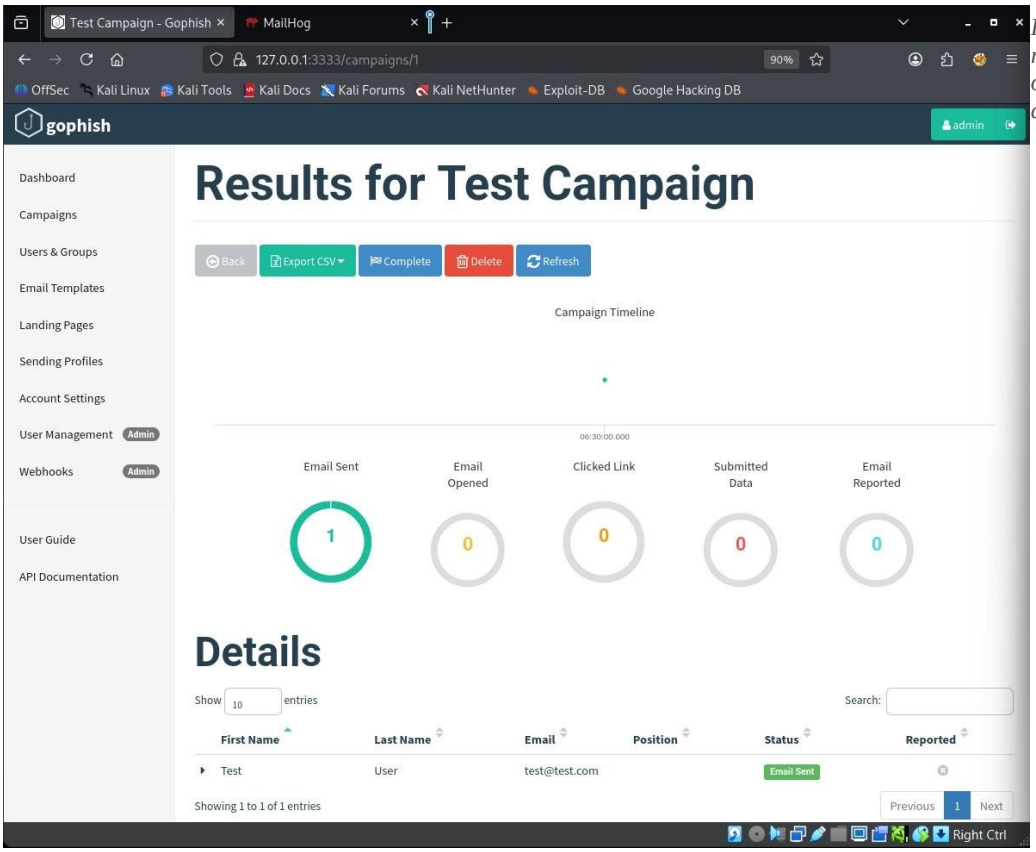


Figure 17 – Campaign results: 1 email sent, 0 opened/clicked (initial campaign)

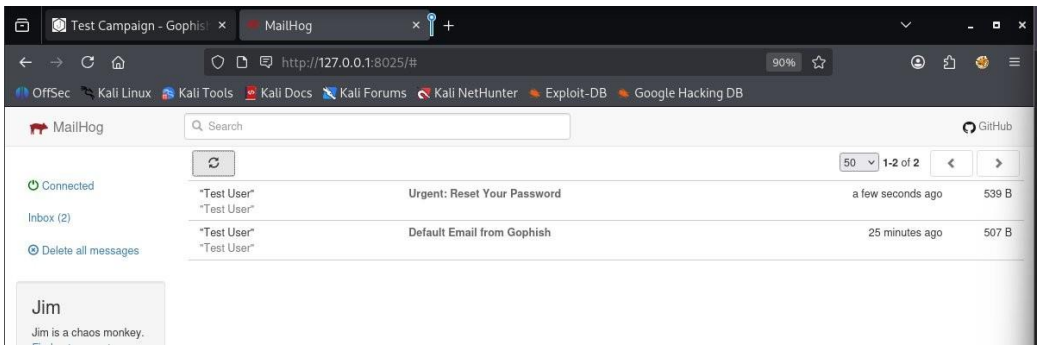


Figure 18 – MailHog inbox showing "Urgent: Reset Your Password" email alongside the earlier test email

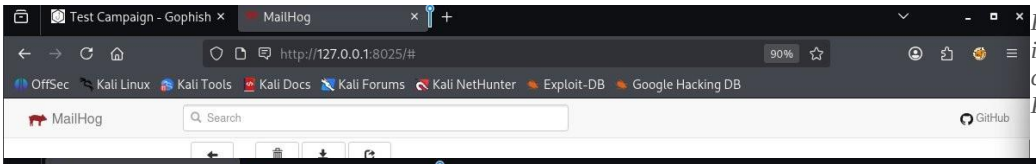


Figure 19 – Email body in MailHog: phishing content with "Reset Password" link

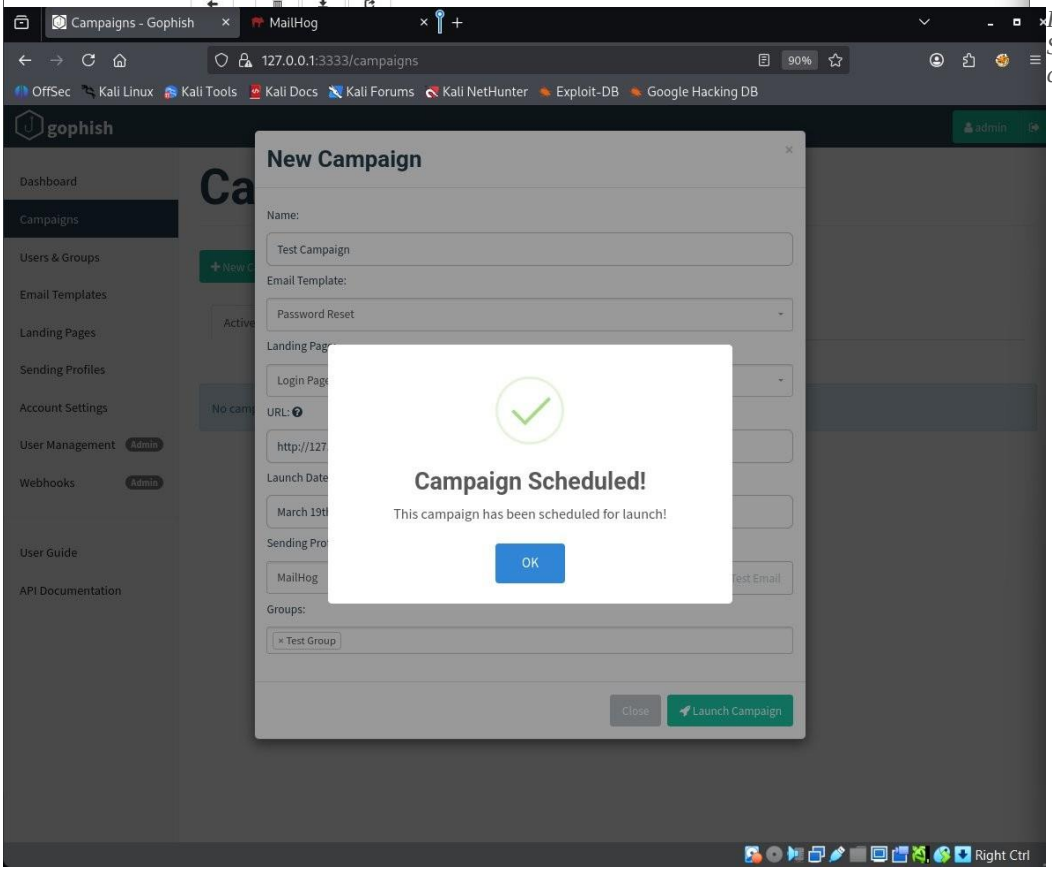


Figure 20 – Campaign Scheduled confirmation dialog

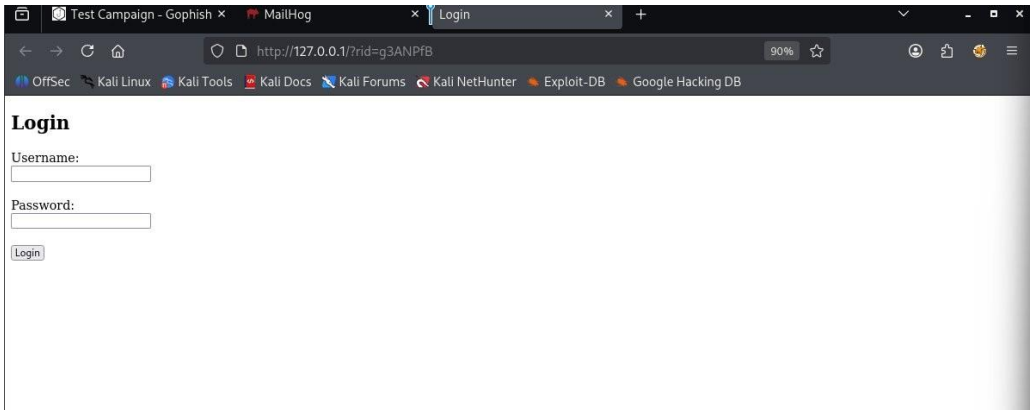


Figure 21 – Phishing landing page rendered at http://127.0.0.1/?rid=g3ANPfB

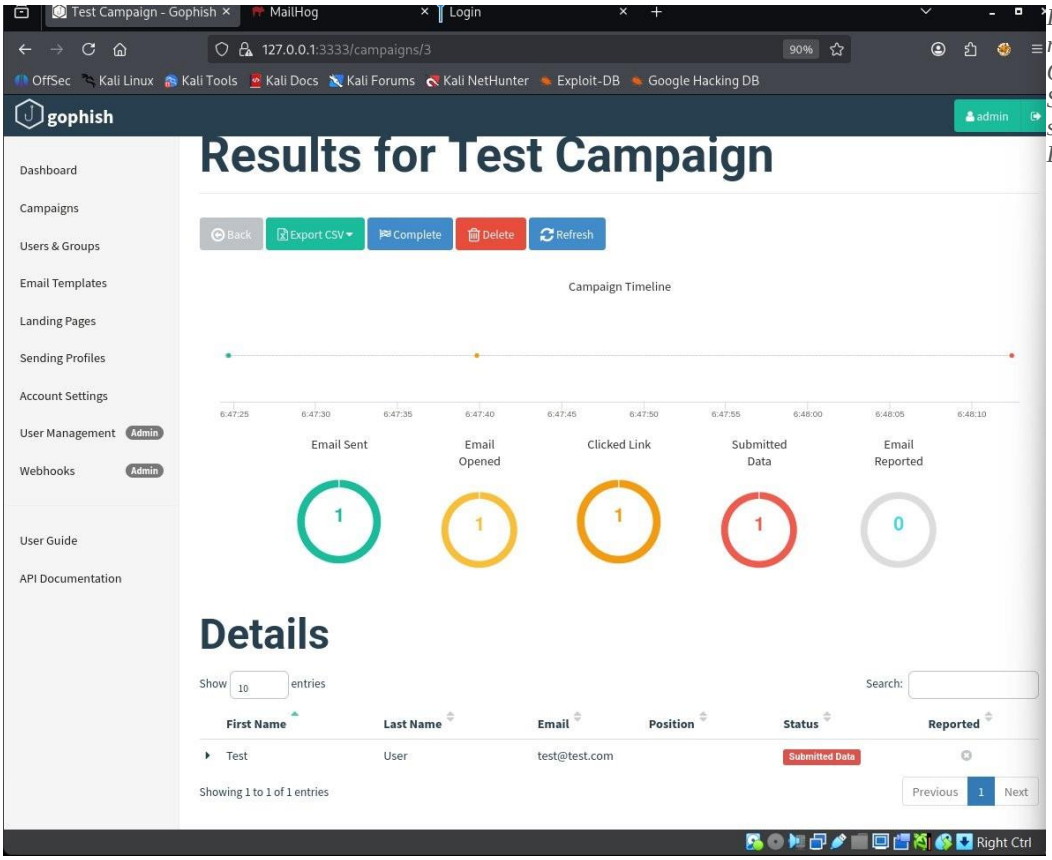


Figure 22 – Campaign results: Email Sent 1, Opened 1, Clicked 1, Submitted Data 1 – status "Submitted Data"